

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Описание предметной области.....	6
1.2 Обзор существующих решений для электронной коммерции	7
1.3 Анализ решений с интеллектуальным подбором товаров.....	9
1.4 Постановка задачи разработки мобильного приложения.....	11
1.5 Обоснование необходимости разработки системы	15
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	16
2.1 Общая архитектура системы	16
2.2 Проектирование мобильного приложения	18
2.3 Проектирование серверной части	20
2.4 Проектирование административной панели	23
2.5 Проектирование базы данных	25
3 РАЗРАБОТКА СИСТЕМЫ	30
3.1 Реализация функциональных возможностей мобильного приложения	30
3.2 Реализация пользовательского интерфейса и навигации	33
3.3 Реализация серверной части	38
3.4 Реализация базы данных	43
3.5 Реализация административной панели	46
3.6 Интеграция внешних сервисов.....	49
4 ТЕСТИРОВАНИЕ СИСТЕМЫ.....	54
5 РАСЧЕТ ТЕХНИКО-ЭКОНОМИЧЕСКИХ ПОКАЗАТЕЛЕЙ.....	68
5.1 Исходные данные для осуществления расчета.....	68
5.2 Расчет объема функций.....	68
5.3 Расчет полной себестоимости программного модуля.....	70
5.4 Расчет отпускной цены и чистой прибыли	74
ЗАКЛЮЧЕНИЕ.....	77
СПИСОК СОКРАЩЕНИЙ	78
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	79
ПРИЛОЖЕНИЕ А. ТЕКСТ ПРОГРАММЫ	

					ДП.АС64.220038 – 06 81 00			
Изм.	Лист	№ докум.	Подпись	Дата				
Разработал	Горкавчук Н.М.				Проектирование и реализация клиент-серверного приложения для электронной коммерции. Пояснительная записка	Стади	Лист	Листов
Проверил	Войцехович Г.Ю.					Д	4	79
Реценз.	Лапич С.В.					УО БрГТУ		
Н. контроль	Войцехович Г.Ю.							
Утвердил	Головки В. А.							

ВВЕДЕНИЕ

Электронная коммерция занимает важное место в современной цифровой среде, а мобильные приложения становятся одним из основных инструментов взаимодействия пользователя с онлайн-магазином. Для покупателя важны удобство поиска товаров, простота оформления заказа, безопасность оплаты и общее качество работы системы.

В условиях большого ассортимента товаров традиционных средств поиска и фильтрации не всегда достаточно для быстрого выбора подходящей продукции. По этой причине особую актуальность приобретают решения, использующие интеллектуальные методы подбора товаров на основе пользовательских запросов, сформулированных в свободной форме.

В рамках данной дипломной работы рассматривается разработка системы электронной коммерции с акцентом на мобильное приложение для покупателей. Проектируемая система включает мобильный клиент, серверную часть и административную панель. Мобильное приложение предоставляет пользователю возможность просматривать каталог, искать товары, добавлять их в корзину и оформлять заказ, серверная часть обеспечивает обработку данных и бизнес-логику, а административная панель предназначена для управления товарами, заказами и клиентами.

Особенностью разрабатываемой системы является наличие модуля интеллектуального подбора товаров, который позволяет улучшить процесс поиска и сделать взаимодействие с приложением более удобным и персонализированным. Использование такого подхода повышает практическую ценность системы и соответствует современным тенденциям развития цифровых торговых решений.

Целью дипломной работы является проектирование и разработка мобильного приложения электронной коммерции с модулем интеллектуального подбора товаров, а также серверной и административной частей системы. Структура работы включает анализ предметной области, проектирование, разработку, тестирование системы и расчет технико-экономических показателей.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Описание предметной области

Электронная коммерция представляет собой сферу экономической деятельности, в рамках которой процессы продажи и покупки товаров или услуг осуществляются с использованием цифровых технологий и сетевых коммуникаций. Основой электронной коммерции являются интернет-платформы, обеспечивающие взаимодействие между продавцом и покупателем, обработку заказов, проведение платежей, организацию доставки и сопровождение клиента на всех этапах покупки. В современных условиях электронная коммерция является одним из наиболее динамично развивающихся направлений цифровой экономики, поскольку позволяет существенно расширить рынок сбыта, повысить доступность товаров и упростить процесс совершения покупок.

Предметная область электронной коммерции включает совокупность объектов, процессов и участников, обеспечивающих функционирование онлайн-торговли. К основным участникам относятся покупатели, продавцы, администраторы системы, а также внешние сервисы, связанные с оплатой, доставкой, авторизацией и обработкой уведомлений. В качестве основных объектов предметной области можно выделить товары, категории товаров, пользователей, корзину, заказы, способы оплаты, адреса доставки, отзывы и рейтинги. Все перечисленные сущности находятся во взаимосвязи и образуют единую цифровую экосистему, в которой каждый этап взаимодействия пользователя с платформой фиксируется и обрабатывается системой.

Ключевым процессом в электронной коммерции является пользовательский сценарий покупки. Как правило, он начинается с авторизации или входа в систему, после чего пользователь получает доступ к каталогу товаров, может выполнять поиск, фильтрацию и сортировку позиций, просматривать карточки товаров, изучать характеристики, изображения, стоимость и отзывы. После выбора необходимых товаров пользователь добавляет их в корзину, указывает адрес доставки, выбирает способ оплаты и оформляет заказ. Завершающими этапами являются подтверждение оплаты, изменение статусов заказа, получение товара и, при необходимости, оставление отзыва. Таким образом, электронная коммерция охватывает не только сам момент покупки, но и полный жизненный цикл взаимодействия клиента с торговой платформой.

Отдельное значение в данной предметной области имеет мобильная коммерция, являющаяся частью электронной коммерции и ориентированная на использование мобильных устройств. Рост популярности смартфонов привёл к тому,

что значительная часть пользователей предпочитает совершать покупки именно через мобильные приложения. Это предъявляет дополнительные требования к разработке систем: интерфейс должен быть интуитивно понятным, навигация — быстрой и логичной, а основные действия — максимально упрощёнными. Кроме того, мобильное приложение должно обеспечивать стабильную работу с каталогом, корзиной, заказами и платежами, а также поддерживать высокий уровень производительности и безопасности при передаче пользовательских данных.

С практической точки зрения современная система электронной коммерции представляет собой не только клиентское приложение для покупателей, но и комплекс взаимосвязанных программных компонентов. Помимо пользовательского интерфейса, важную роль играет серверная часть, отвечающая за обработку бизнес-логики, хранение и валидацию данных, управление заказами, взаимодействие с платёжными системами и внешними сервисами. Не менее значимым элементом является административная панель, предназначенная для управления товарами, заказами, пользователями и аналитическими показателями. Следовательно, предметная область электронной коммерции охватывает как пользовательские процессы, так и внутренние процессы управления торговой платформой.

Современные тенденции развития электронной коммерции связаны с персонализацией пользовательского опыта, интеллектуальной обработкой запросов и повышением качества подбора товаров. Традиционные методы поиска по точному совпадению названия или фильтров уже не всегда удовлетворяют потребности пользователей, поскольку покупатель нередко формулирует запрос в свободной форме, опираясь не на точное наименование товара, а на желаемые характеристики и сценарий использования. В связи с этим всё большую актуальность приобретают решения, использующие элементы искусственного интеллекта для анализа пользовательских запросов, интерпретации предпочтений и формирования более релевантных рекомендаций. Такие механизмы позволяют повысить удобство взаимодействия с системой и увеличить вероятность успешного совершения покупки.

1.2 Обзор существующих решений для электронной коммерции

В настоящее время мобильные приложения электронной коммерции занимают важное место в цифровой торговле, обеспечивая пользователям постоянный доступ к товарам, услугам и сопутствующим функциям через смартфоны и планшеты. Развитие мобильных технологий, рост популярности онлайн-покупок и стремление пользователей к быстрому и удобному

Изм	Лист	№ докум	Подп.	Дата

ДП.АС64.220038 – 06 81 00

Лист

7

взаимодействию с торговыми платформами способствовали широкому распространению мобильных решений в данной области. Современные приложения позволяют не только просматривать каталог товаров, но и выполнять поиск, сравнение, оформление заказа, оплату, отслеживание доставки и взаимодействие с продавцом в рамках единой цифровой среды.

Среди наиболее известных приложений электронной коммерции, используемых русскоязычными пользователями, можно выделить Wildberries, Ozon и Onliner. Эти платформы предлагают широкий ассортимент товаров, охватывающий как повседневные категории, так и более специализированные группы продукции. Их мобильные приложения ориентированы на массового пользователя и предоставляют стандартный набор функций, включающий каталог товаров, систему поиска, фильтрацию, корзину, оформление заказа, выбор способа доставки, оплату и доступ к личному кабинету. За счёт этого они формируют для пользователя привычную модель взаимодействия с цифровой торговой системой.

Приложения Wildberries и Ozon представляют собой крупные маркетплейсы, в которых пользователь получает доступ к большому количеству товаров от разных продавцов. Их основным преимуществом является широкий выбор продукции в рамках одного сервиса, что позволяет удовлетворять разнообразные потребности без необходимости перехода между несколькими платформами. Пользователь может выбирать товары по категориям, подкатегориям, брендам, ценовому диапазону, рейтингу, срокам доставки и другим параметрам. Подобная организация интерфейса делает процесс покупки достаточно привычным и удобным, особенно в тех случаях, когда пользователь уже понимает, какой именно товар ему необходим.

Особое место среди подобных решений занимает Onliner, который сочетает функции торговой площадки, каталога товаров и сервиса сравнения предложений. В его структуре значительное внимание уделяется техническим характеристикам товаров, отзывам, рейтингу, обзорам и сравнению параметров. Такой подход особенно полезен при выборе электроники и других технически сложных товаров, где решение о покупке нередко принимается на основе анализа детальных спецификаций. Вместе с тем именно в таких категориях становится заметной одна из существенных проблем существующих систем электронной коммерции.

Несмотря на широкий функционал и высокую степень развития мобильных приложений, подбор товаров в большинстве случаев по-прежнему строится на основе классического поиска и набора фильтров. Пользователь должен самостоятельно определить категорию товара, выбрать нужные параметры и сузить выбор за счёт последовательного применения фильтров. Такой подход является эффективным, если покупатель уже знает точное наименование товара или хорошо ориентируется в его характеристиках. Однако на практике многие пользователи формулируют свою потребность не через точные технические параметры, а через

повседневные описания, например: «нужны удобные наушники для спорта», «хочу игровой смартфон», «нужен недорогой ноутбук для учёбы и работы». В подобных случаях стандартные фильтры не всегда позволяют быстро получить действительно подходящий результат.

Дополнительная сложность заключается в том, что часть характеристик товаров, особенно в категориях электроники, бытовой техники и аксессуаров, может быть непонятна обычному пользователю. Такие параметры, как тип матрицы экрана, частота обновления, модель процессора, объём оперативной памяти, стандарты защиты, тип интерфейса подключения или особенности аккумулятора, зачастую требуют определённого уровня технической подготовки. Для пользователя, не обладающего специальными знаниями, выбор по подобным критериям становится затруднительным. В результате процесс покупки может усложняться, а принятие решения — откладываться или становиться менее обоснованным.

Дополнительно следует отметить, что эффективность мобильного приложения электронной коммерции определяется не только широтой ассортимента и наличием базовых функций, но и тем, насколько быстро пользователь может перейти от возникновения потребности к оформлению заказа. В условиях большого количества товаров и высокой конкуренции между цифровыми площадками особую значимость приобретают удобство интерфейса, логичность навигации, качество поиска и способность системы сокращать число действий, необходимых для выбора подходящей позиции. Если приложение требует от пользователя глубокого понимания характеристик товара, ручного перебора большого количества фильтров и самостоятельного сравнения множества вариантов, это усложняет процесс покупки и снижает общее качество пользовательского опыта. В связи с этим современные мобильные решения в сфере электронной коммерции всё чаще рассматриваются не только как средство доступа к каталогу товаров, но и как инструмент поддержки принятия решения о покупке, что делает особенно актуальным внедрение механизмов интеллектуального подбора продукции.

1.3 Анализ решений с интеллектуальным подбором товаров

В современных системах электронной коммерции всё более заметное место занимают решения, ориентированные на интеллектуальный подбор товаров. В отличие от традиционного поиска по ключевым словам и фильтрам, такие механизмы стремятся учитывать смысл пользовательского запроса, контекст покупки, индивидуальные предпочтения и сценарий использования товара. Основная цель подобных решений заключается в том, чтобы упростить процесс

выбора продукции, сократить время поиска и повысить вероятность того, что пользователь найдёт товар, действительно соответствующий его потребности. Развитие данного направления связано с ростом объёма электронных каталогов и усложнением выбора, особенно в категориях, где покупателю приходится учитывать большое количество характеристик и параметров.

Одним из наиболее заметных направлений являются диалоговые интеллектуальные ассистенты, встроенные в торговые платформы. Их отличие от обычного поискового интерфейса состоит в том, что пользователь может формулировать запрос в естественной форме, не зная точного названия товара или его технических параметров. В качестве примера можно привести Amazon Rufus, который, по официальному описанию Amazon, способен искать товары по событию, активности, назначению и иным сценариям использования, отвечать на вопросы о товарах, помогать в сравнении вариантов и поддерживать пользователя в процессе выбора. Такой подход показывает переход от модели «пользователь сам ищет нужный товар» к модели «система помогает интерпретировать потребность и предлагает релевантные решения».

Другим важным направлением является интеллектуальный поиск по естественному языку и контексту запроса. В подобных системах анализируется не только набор введённых слов, но и общий смысл запроса, а также дополнительные факторы, влияющие на решение о покупке. Google описывает развитие shopping-функций в AI Mode как механизм, который формирует структурированный ответ с визуальными элементами, сведениями о цене, отзывах и наличии товара, помогая пользователю быстрее принять решение. Кроме того, платформа развивает функции сравнения вариантов и визуального подбора, в том числе виртуальную примерку одежды с использованием фотографии пользователя. Это свидетельствует о том, что современные решения всё чаще сочетают текстовый анализ запроса, контекстные рекомендации и визуальные инструменты поддержки выбора.

Отдельную группу составляют персонализированные и агентные механизмы подбора, в которых система не просто реагирует на разовый запрос, а стремится сопровождать пользователя в процессе выбора. По официальному описанию eBay, agentic AI используется для предоставления гиперперсонализированных товарных рекомендаций и экспертных подсказок непосредственно в ходе взаимодействия с маркетплейсом. Подобные решения демонстрируют движение электронной коммерции в сторону более активной роли системы, когда платформа не ограничивается выдачей списка товаров, а помогает пользователю уточнить потребность, понять различия между вариантами и сузить круг выбора.

Анализ существующих решений показывает, что интеллектуальный подбор товаров в настоящее время развивается по нескольким основным направлениям: диалоговое взаимодействие с пользователем, семантический поиск по

естественному языку, персонализированные рекомендации и визуальные средства поддержки выбора. Все эти подходы направлены на решение общей проблемы, характерной для традиционных систем электронной коммерции, а именно высокой зависимости от ручного выбора категорий и фильтров. В классической модели пользователь должен самостоятельно понимать, какие параметры товара являются значимыми, и уметь интерпретировать технические характеристики. Интеллектуальные решения частично снимают эту нагрузку, поскольку позволяют формулировать запрос более естественным образом и получать рекомендации, ориентированные не только на формальные признаки, но и на предполагаемый сценарий использования товара.

Вместе с тем существующие решения имеют и ряд ограничений. Во-первых, наиболее развитые механизмы интеллектуального подбора реализуются, как правило, в рамках крупных международных платформ, располагающих значительными массивами пользовательских и товарных данных. Во-вторых, часть функций внедряется постепенно и может быть ограничена по региону, языку или категории товаров; например, официальные материалы Amazon и Google прямо указывают на запуск отдельных shopping-функций для пользователей в США. Во-третьих, даже при наличии интеллектуального помощника качество рекомендаций по-прежнему зависит от полноты и корректности товарных данных, а окончательное решение о покупке остаётся за пользователем. Следовательно, интеллектуальный подбор не заменяет полностью традиционные механизмы выбора, но позволяет сделать их более удобными, понятными и ориентированными на реальные потребности покупателя.

Таким образом, проведённый анализ показывает, что наиболее перспективным направлением развития электронной коммерции является сочетание нескольких подходов: обработки запросов на естественном языке, анализа пользовательского контекста, сопоставления потребности с характеристиками товаров и формирования персонализированных рекомендаций.

1.4 Постановка задачи разработки мобильного приложения

На основании проведённого анализа предметной области электронной коммерции, обзора существующих мобильных приложений и рассмотрения решений с интеллектуальным подбором товаров можно сделать вывод о

необходимости разработки современной программной системы, ориентированной на упрощение процесса выбора и покупки товаров в мобильной среде. Разрабатываемая система должна представлять собой не отдельный программный модуль, а комплексное решение, включающее мобильное приложение для покупателей, серверную часть, обеспечивающую бизнес-логику и обработку данных, а также административную панель для управления содержимым и основными процессами торговой платформы.

Основной задачей разработки является создание мобильного приложения электронной коммерции, которое позволит пользователю выполнять полный цикл взаимодействия с системой: проходить авторизацию, просматривать каталог товаров, осуществлять поиск и фильтрацию, добавлять товары в избранное и корзину, оформлять заказ, выбирать адрес доставки, оплачивать покупку и оставлять отзыв после получения товара. При этом система должна обеспечивать удобный пользовательский интерфейс, логичную навигацию, высокую скорость взаимодействия и надёжную обработку пользовательских действий.

Особенностью разрабатываемого решения является наличие модуля интеллектуального подбора товаров. В отличие от традиционного поиска, основанного только на категориях и фильтрах, данный модуль должен обеспечивать возможность обработки пользовательских запросов, сформулированных в свободной форме, и подбирать релевантные товары на основе анализа текста запроса, характеристик каталога и механизмов интеллектуального ранжирования. Такой подход должен повысить удобство использования системы, особенно в случаях, когда пользователь не знает точного наименования товара или не ориентируется в его технических параметрах.

Для достижения поставленной цели в рамках дипломного проекта необходимо решить ряд взаимосвязанных задач: разработать архитектуру системы; спроектировать мобильное приложение, серверную часть, административную панель и структуру базы данных; реализовать функциональные возможности пользовательского приложения; обеспечить обработку заказов, оплат и пользовательских данных на стороне сервера; создать средства администрирования товаров, заказов и клиентов; разработать модуль интеллектуального подбора товаров; выполнить интеграцию внешних сервисов, необходимых для авторизации, оплаты, хранения данных и поддержки интеллектуального поиска; а также провести тестирование разработанной системы и оценить результаты её работы. Указанные задачи соответствуют общей структуре дипломного проекта и определяют содержание последующих разделов, посвящённых проектированию, разработке и тестированию программного продукта.

Таким образом, постановка задачи разработки мобильного приложения заключается в создании комплексной системы электронной коммерции,

обеспечивающей удобный пользовательский сценарий покупки, эффективное управление торговыми процессами и применение интеллектуальных механизмов подбора товаров. Результатом работы должна стать функционирующая программная система, способная повысить удобство поиска продукции, сократить время принятия решения о покупке и улучшить общее качество взаимодействия пользователя с электронной торговой платформой.

При постановке задачи разработки необходимо также учитывать требования к надёжности, масштабируемости и практической применимости создаваемой системы. Разрабатываемое мобильное приложение должно обеспечивать корректную работу не только пользовательского интерфейса, но и всех взаимосвязанных компонентов, включая серверную часть, обработку заказов, управление каталогом, оплату и административные функции. Особое значение имеет возможность расширения системы за счёт добавления новых категорий товаров, характеристик, пользовательских сценариев и механизмов интеллектуального подбора без необходимости полного изменения архитектуры решения. Такой подход позволяет рассматривать разрабатываемую систему не как узкоспециализированное приложение, а как основу для дальнейшего развития полноценной электронной торговой платформы.

Необходимость разработки рассматриваемой системы обусловлена особенностями современного рынка электронной коммерции, в котором пользователь ожидает не только широкого ассортимента товаров, но и простого, быстрого и удобного процесса выбора и покупки. При большом количестве товарных позиций традиционные механизмы поиска по категориям, ключевым словам и фильтрам не всегда обеспечивают получение действительно релевантного результата. Особенно это проявляется в тех случаях, когда пользователь не знает точного названия товара, не ориентируется в технических характеристиках или формулирует свой запрос в естественной форме, исходя не из параметров продукции, а из собственной потребности.

Существующие мобильные приложения электронной коммерции в целом успешно решают базовые задачи онлайн-торговли: предоставляют доступ к каталогу, поддерживают корзину, оформление заказа, оплату и отслеживание доставки. Однако во многих случаях процесс подбора товара остаётся недостаточно гибким и требует от пользователя самостоятельного анализа большого числа характеристик и альтернатив. Это приводит к увеличению времени выбора, усложняет принятие решения о покупке и может снижать общее качество пользовательского опыта. Следовательно, возникает потребность в системе, которая будет не только отображать каталог товаров, но и активно помогать пользователю в подборе наиболее подходящих позиций.

Дополнительным фактором, обосновывающим необходимость разработки системы, является широкое распространение мобильных устройств как основного средства взаимодействия с цифровыми сервисами. Для значительной части пользователей именно мобильное приложение является наиболее удобным каналом доступа к торговой платформе. В связи с этим особую актуальность приобретает создание решения, в котором основные функции электронной коммерции будут доступны в мобильной форме, а интерфейс и логика взаимодействия будут адаптированы к повседневным пользовательским сценариям. Разработка мобильного приложения в рамках данной системы позволяет обеспечить доступность сервиса, сократить количество действий при совершении покупки и повысить удобство взаимодействия с каталогом, заказами и оплатой.

Необходимость разработки системы обусловлена также потребностями со стороны управления торговой платформой. Для полноценного функционирования электронного магазина недостаточно только пользовательского приложения, так как требуется централизованное управление товарами, их характеристиками, заказами, клиентами и аналитическими данными. Поэтому разрабатываемая система должна включать не только мобильный клиент, но и серверную часть, обеспечивающую хранение и обработку данных, реализацию бизнес-логики и интеграцию внешних сервисов, а также административную панель, предназначенную для сопровождения торговых процессов. Такой комплексный подход позволяет создать целостное программное решение, пригодное для практического применения.

Особую значимость в рамках разрабатываемой системы имеет модуль интеллектуального подбора товаров. Его необходимость определяется тем, что современные пользователи всё чаще ожидают от цифровых сервисов более естественного и персонализированного взаимодействия. Возможность формулировать запрос в свободной форме и получать рекомендации, основанные на анализе потребности, делает процесс выбора более понятным и доступным. Для электронной коммерции это особенно важно в категориях товаров со сложными или многочисленными характеристиками. Введение интеллектуального подбора позволяет снизить зависимость пользователя от ручного применения фильтров, упростить поиск релевантной продукции и тем самым повысить эффективность работы всей системы.

Таким образом, необходимость разработки системы определяется совокупностью нескольких факторов: потребностью в удобном мобильном инструменте для онлайн-покупок, необходимостью повышения качества подбора товаров, важностью централизованного управления торговыми процессами и актуальностью внедрения интеллектуальных механизмов поддержки выбора. Разработка такой системы является обоснованной, поскольку она направлена на решение реальных пользовательских и организационных задач и соответствует

общей структуре дипломного проекта, включающей проектирование, разработку, тестирование и экономическое обоснование программного продукта.

1.5 Постановка задачи разработки мобильного приложения

Целью разработки является создание мобильного приложения для электронной коммерции, которое позволит пользователю удобно просматривать каталог товаров, получать подробную информацию о выбранных позициях, добавлять товары в корзину и избранное, оформлять заказы, выбирать адрес доставки и выполнять оплату. Приложение должно быть ориентировано на простой и понятный пользовательский сценарий, чтобы процесс поиска и покупки товара не требовал от пользователя лишних действий.

В рамках поставленной задачи необходимо реализовать мобильный клиент, связанный с серверной частью, базой данных и внешними сервисами авторизации, оплаты и обработки изображений. Отдельное внимание уделяется модулю интеллектуального подбора товаров, который должен позволять пользователю формулировать запрос в свободной форме, например описывать желаемый товар обычными словами, после чего система должна подбирать наиболее подходящие позиции из каталога. Такая функциональность делает приложение более удобным по сравнению с обычным поиском и фильтрацией.

Дополнительно в процессе разработки необходимо обеспечить стабильность и безопасность работы приложения. Для этого требуется предусмотреть защищенную авторизацию пользователей, проверку данных при оформлении заказа, корректную обработку ошибок и сохранение актуального состояния корзины, заказов и профиля пользователя. Также приложение должно быть достаточно гибким для дальнейшего расширения, например добавления новых категорий товаров, характеристик, способов доставки или дополнительных функций интеллектуального поиска.

Таким образом, разрабатываемое мобильное приложение должно обеспечить полный цикл взаимодействия пользователя с интернет-магазином: от регистрации и выбора товара до оформления заказа, оплаты и последующей оценки покупки. Серверная часть, административная панель и интеграции с внешними сервисами выступают вспомогательными компонентами, обеспечивающими корректную работу мобильного приложения и управление данными системы.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Общая архитектура системы

Разрабатываемая система представляет собой программный комплекс для электронной коммерции, предназначенный для просмотра, выбора, покупки и администрирования товаров. В состав системы входят мобильное приложение для покупателей, серверная часть, база данных, административная панель, а также внешние сервисы, необходимые для авторизации пользователей, обработки платежей, хранения изображений и интеллектуального подбора товаров.

Основой проектируемой системы является клиент-серверная архитектура. Мобильное приложение и административная панель выступают в качестве клиентских частей, а серверная часть выполняет обработку запросов, проверку прав доступа, взаимодействие с базой данных и подключенными внешними сервисами. Такой подход позволяет отделить пользовательский интерфейс от бизнес-логики и централизовать работу с данными.

Мобильное приложение является основной пользовательской частью системы. Оно предназначено для покупателей и обеспечивает доступ к каталогу товаров, поиску, фильтрации, карточкам товаров, корзине, избранному, оформлению заказа, оплате, истории заказов и профилю пользователя. Для разработки мобильной части используется React Native, так как данная технология позволяет создавать мобильные приложения на основе React и разрабатывать интерфейс для разных мобильных платформ [4].

Серверная часть является центральным элементом архитектуры. Она принимает запросы от мобильного приложения и административной панели, обрабатывает их и возвращает необходимые данные. На сервере реализуется основная бизнес-логика: работа с пользователями, получение и фильтрация товаров, управление корзиной, оформление заказов, проверка стоимости товаров, обработка платежей, работа с отзывами и взаимодействие с модулем интеллектуального подбора. Для разработки серверной API-части используется Express, который является веб-фреймворком для Node.js и подходит для создания API веб- и мобильных приложений [5].

Административная панель предназначена для управления интернет-магазином со стороны администратора. С ее помощью выполняется добавление и редактирование товаров, управление их изображениями и характеристиками, просмотр заказов, изменение статусов заказов, работа с клиентами и анализ основных статистических показателей. Наличие отдельной административной панели позволяет разделить функции покупателя и функции управления системой.

База данных используется для хранения основной информации, необходимой для работы приложения. В ней сохраняются сведения о пользователях, товарах, категориях, подкатегориях, характеристиках товаров, корзинах, адресах доставки, заказах, платежах, отзывах и других сущностях. Для хранения данных используется MongoDB, так как документоориентированная модель удобна для работы с товарами, которые могут иметь разные наборы характеристик в зависимости от категории и подкатегории [6].

В архитектуре системы можно выделить несколько основных компонентов. Мобильное приложение используется покупателем для просмотра каталога, выбора товаров, оформления заказов, оплаты и работы с профилем. Административная панель используется администратором для управления товарами, заказами, клиентами и статистикой. Серверная часть обрабатывает запросы, реализует бизнес-логику, проверяет права доступа и взаимодействует с внешними сервисами. База данных хранит информацию о пользователях, товарах, заказах, адресах, отзывах и других сущностях. Сервис авторизации обеспечивает регистрацию, вход пользователей и защиту маршрутов. Платежный сервис используется для обработки оплаты заказов. Сервис хранения изображений применяется для загрузки и хранения изображений товаров. Модуль интеллектуального подбора товаров обрабатывает пользовательские запросы и подбирает релевантные товары из каталога.

Взаимодействие покупателя с системой начинается с авторизации или регистрации в мобильном приложении. После входа пользователь получает доступ к каталогу товаров, поиску, фильтрам и карточкам товаров. При оформлении заказа данные из корзины передаются на серверную часть, где выполняется проверка актуальности товаров, их стоимости и состава заказа. Это необходимо для того, чтобы критически важные операции не выполнялись только на стороне клиентского приложения. После проверки сервер взаимодействует с платежным сервисом, а информация о заказе сохраняется в базе данных после подтверждения оплаты.

Отдельное место в системе занимает модуль интеллектуального подбора товаров. Его задача заключается в том, чтобы упростить процесс выбора товара для пользователя. Вместо самостоятельного выбора точных параметров и фильтров пользователь может описать свою потребность обычным текстом. Серверная часть анализирует такой запрос, сопоставляет его с товарами из каталога и формирует список наиболее подходящих вариантов. Для реализации интеллектуальной обработки запроса может использоваться Gemini API, предоставляющий доступ к моделям Gemini через API [7].

Важной особенностью архитектуры является разграничение доступа. Обычный пользователь получает доступ к функциям, связанным с просмотром и покупкой товаров, а администратор — к функциям управления товарами, заказами, клиентами и статистикой. Проверка прав доступа выполняется на серверной части,

что позволяет защитить административные функции от несанкционированного использования.

Таким образом, выбранная архитектура обеспечивает логическое разделение компонентов системы и их обязанностей. Мобильное приложение отвечает за взаимодействие с покупателем, административная панель — за управление интернет-магазином, серверная часть — за бизнес-логику и безопасность, база данных — за хранение информации, а внешние сервисы — за выполнение специализированных функций. Такое построение системы удобно для разработки, сопровождения и дальнейшего масштабирования проекта.

2.2 Проектирование мобильного приложения

Для разработки мобильного приложения используется React Native, так как данная технология позволяет создавать мобильные приложения на основе React и применять единый подход к разработке интерфейса для разных мобильных платформ [4]. В качестве инструмента разработки применяется Expo, который упрощает запуск, тестирование и сборку React Native-приложений [8]. Навигация между экранами проектируется с использованием Expo Router, основанного на файловой маршрутизации [9].

Пользовательский сценарий начинается с экрана авторизации или регистрации. После успешного входа пользователь получает доступ к основным функциям приложения. Авторизация необходима для сохранения пользовательских данных, работы с корзиной, избранным, адресами доставки, заказами и оплатой. Для реализации авторизации используется Clerk, который предоставляет средства для управления пользователями и сессиями в Expo-приложениях [11].

Главный экран приложения предназначен для быстрого перехода к основным разделам системы. На нем могут отображаться категории, популярные товары, строка поиска и переход к интеллектуальному подбору. Основная задача данного экрана заключается в том, чтобы упростить пользователю начало работы с приложением и направить его к нужному разделу.

Каталог товаров является одним из ключевых экранов мобильного приложения. Он предназначен для отображения списка товаров и предоставляет пользователю инструменты поиска, фильтрации и сортировки. Поскольку товары могут относиться к разным категориям и иметь различные характеристики, интерфейс каталога должен быть гибким и позволять отображать параметры, соответствующие конкретной категории или подкатегории.

Карточка товара содержит подробную информацию о выбранной позиции: изображения, название, цену, описание, характеристики, рейтинг и отзывы. С данного экрана пользователь может добавить товар в корзину, сохранить его в избранное или перейти к оформлению заказа. Карточка товара также может использоваться как результат работы интеллектуального подбора, когда система предлагает пользователю наиболее подходящие варианты.

Общая структура экранов мобильного приложения и основные переходы между ними представлены на рисунке 2.1

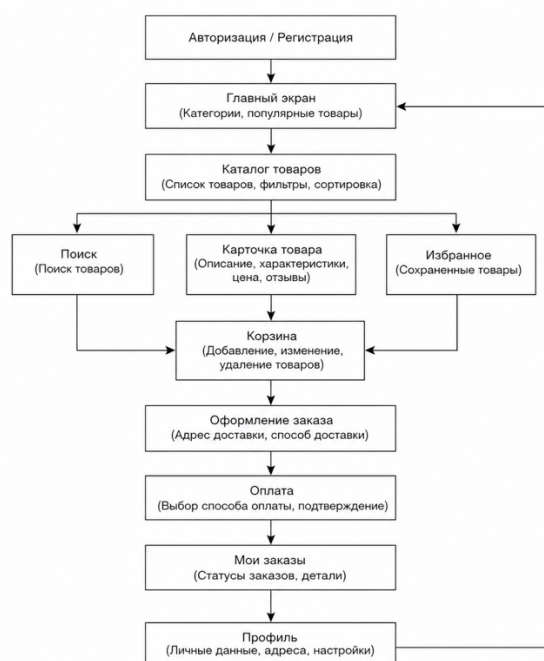


Рисунок 2.1 – Схема структуры мобильного приложения

Корзина предназначена для хранения товаров, выбранных пользователем перед покупкой. В ней пользователь может изменять количество товаров, удалять позиции и проверять итоговую стоимость. При этом окончательная проверка цены, состава заказа и доступности товаров должна выполняться на серверной стороне, а не только в мобильном приложении. Такой подход снижает риск ошибок и защищает критически важные данные от изменения на стороне клиента.

Экран оформления заказа используется для подтверждения состава заказа, выбора адреса доставки и перехода к оплате. Для реализации платежного функционала применяется Stripe React Native SDK, позволяющий встроить оплату в мобильные приложения на React Native [12]. Мобильное приложение отвечает за отображение платежного интерфейса, а создание платежа, проверка заказа и подтверждение оплаты выполняются серверной частью.

Профиль пользователя включает личные данные, адреса доставки, историю заказов и настройки аккаунта. В истории заказов пользователь может просматривать ранее оформленные заказы, их состав, стоимость и текущий статус. После доставки

товара пользователь может оставить отзыв или оценку, что позволяет связать пользовательские оценки с реально выполненными заказами.

Отдельным элементом мобильного приложения является модуль интеллектуального подбора товаров. Он позволяет пользователю искать товары не только с помощью обычных фильтров, но и через запрос в свободной форме. Например, пользователь может описать, какой товар ему нужен, а система подберет подходящие варианты и отобразит их в виде карточек товаров. В проекте предусмотрен гибридный подход: сначала выполняется локальный поиск и ранжирование по каталогу, после чего Gemini может использоваться для уточнения рекомендаций.

Для работы с серверными данными используется TanStack Query. Данная библиотека позволяет организовать загрузку, кэширование и обновление данных, что важно для каталога, корзины, избранного, профиля и истории заказов [10]. Для стилизации интерфейса применяется NativeWind, позволяющий использовать подход Tailwind CSS в React Native-приложении [13]. Для мониторинга ошибок может использоваться Sentry, поддерживающий отслеживание исключений в React Native-приложениях [14].

2.3 Проектирование серверной части

Отдельным элементом мобильного приложения является модуль интеллектуального подбора товаров. Он позволяет пользователю искать товары не только с помощью обычных фильтров, но и через запрос в свободной форме. Например, пользователь может описать, какой товар ему нужен, а система подберет подходящие варианты и отобразит их в виде карточек товаров. В проекте предусмотрен гибридный подход: сначала выполняется локальный поиск и ранжирование по каталогу, после чего Gemini может использоваться для уточнения рекомендаций.

Серверная часть является центральным элементом разрабатываемой системы электронной коммерции. Она обеспечивает связь между мобильным приложением, административной панелью, базой данных и внешними сервисами. Через backend выполняются основные бизнес-процессы: авторизация пользователей, получение товаров, работа с корзиной, оформление заказов, обработка платежей, загрузка изображений, управление отзывами и взаимодействие с модулем интеллектуального подбора товаров. В рамках проекта backend связывает клиентские приложения, бизнес-правила, интеграции и хранение данных в единую систему .

Для разработки серверной части используется Node.js и Express. Express применяется для создания API-маршрутов, обработки HTTP-запросов и передачи данных между мобильным приложением, административной панелью и сервером [5]. Серверная часть строится по модульному принципу, что позволяет разделить функциональность на отдельные логические блоки: пользователи, товары, категории, корзина, заказы, платежи, отзывы, изображения, административные функции и интеллектуальный подбор товаров.

Модуль авторизации отвечает за проверку пользователя и защиту закрытых маршрутов. Для этого используется Clerk middleware. Обычные пользователи получают доступ к функциям мобильного приложения, связанным с просмотром и покупкой товаров, а администраторы — к управлению товарами, заказами, клиентами и статистикой.

Модуль товаров обеспечивает получение списка товаров, просмотр карточки товара, поиск, фильтрацию и сортировку. Для административной панели он также должен поддерживать создание, редактирование и удаление товаров. Так как товары могут относиться к разным категориям и иметь разные характеристики, сервер должен корректно обрабатывать динамический набор параметров товара.

Для хранения данных используется MongoDB. Данная база данных хранит информацию в виде документов и коллекций, что удобно для работы с товарами, пользователями, заказами, корзинами, адресами и отзывами [6]. Для взаимодействия серверного приложения с MongoDB используется Mongoose, позволяющий описывать модели данных, схемы и правила валидации.

Модуль корзины отвечает за добавление товаров, изменение их количества, удаление позиций и получение текущего состояния корзины пользователя. При этом итоговая стоимость заказа не должна определяться только на стороне мобильного приложения. Серверная часть должна самостоятельно проверять цену, наличие товара и состав заказа перед переходом к оплате.

Модуль заказов отвечает за создание заказа, хранение его состава, статуса, адреса доставки, суммы и данных пользователя. При оформлении заказа сервер получает данные из корзины, проверяет их актуальность и формирует информацию для оплаты. Такой подход повышает надежность системы и не позволяет клиентскому приложению самостоятельно изменять критически важные данные заказа.

Модуль платежей проектируется с использованием Stripe. Серверная часть отвечает за создание платежа и обработку результата оплаты. Для подтверждения успешной оплаты используется webhook-подход: платежный сервис отправляет событие на сервер после изменения состояния платежа [15]. Только после получения подтверждения сервер может изменить статус заказа и сохранить информацию об оплате в базе данных.

Модуль загрузки изображений используется для работы с изображениями товаров. При добавлении или редактировании товара административная панель передает изображение на сервер, после чего оно загружается во внешний сервис Cloudinary. В базе данных при этом сохраняется ссылка на изображение, которая затем используется в мобильном приложении и административной панели.

Общая структура серверной части и ее взаимодействие с внешними сервисами представлены на рисунке 2.2.

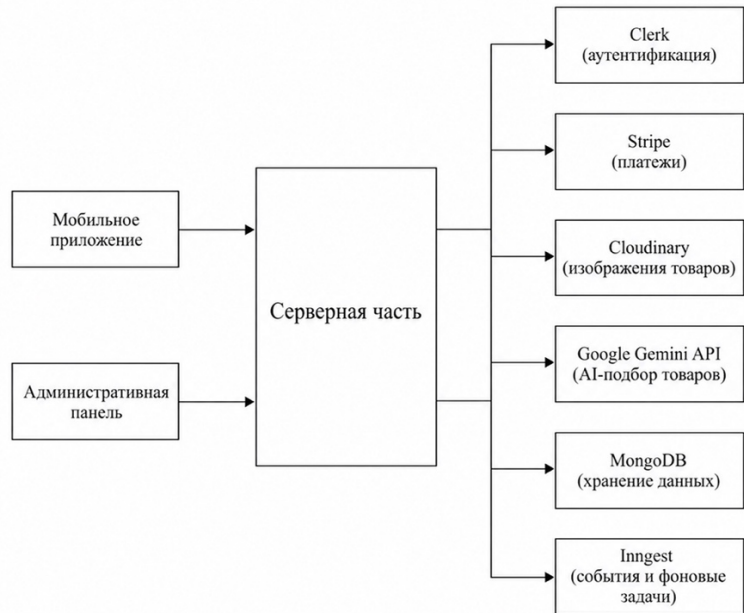


Рисунок 2.2 – Структура серверной части

Стоит отметить, что в рамках серверной части также проектируется модуль интеллектуального подбора товаров. Он принимает пользовательский запрос в свободной форме, например описание нужного товара, и подбирает подходящие позиции из каталога. Сначала сервер может выполнить локальный поиск и ранжирование товаров по названию, описанию, категории, характеристикам и цене. После этого для уточнения запроса и формирования более точной рекомендации может использоваться Gemini API [7].

Такой подход позволяет сохранить устойчивость системы. Если внешний AI-сервис временно недоступен, серверная часть может вернуть результаты локального поиска без полной остановки функции подбора. Благодаря этому интеллектуальный подбор товаров не становится критической точкой отказа, а пользователь продолжает получать подходящие варианты товаров.

Для обработки отдельных событий и фоновых задач может использоваться Inngest. Он может применяться для обработки событий, связанных с пользователями, заказами, уведомлениями или другими асинхронными процессами. Это позволяет не перегружать основные API-запросы дополнительными операциями.

2.4 Проектирование административной панели

Стоит отметить, что в рамках серверной части также проектируется модуль интеллектуального подбора товаров. Он принимает пользовательский запрос в свободной форме, например описание нужного товара, и подбирает подходящие позиции из каталога. Сначала сервер может выполнить локальный поиск и ранжирование товаров по названию, описанию, категории, характеристикам и цене. После этого для уточнения запроса и формирования более точной рекомендации может использоваться Gemini API [7].

Административная панель является отдельной частью разрабатываемой системы электронной коммерции и предназначена для управления интернет-магазином со стороны администратора. В отличие от мобильного приложения, которое ориентировано на покупателей, административная панель используется для выполнения внутренних операций: управления товарами, заказами, клиентами и просмотра статистической информации. Наличие такой панели позволяет администратору контролировать работу магазина без прямого вмешательства в базу данных или серверный код.

Доступ к административной панели должен быть ограничен только для пользователей с соответствующими правами. После авторизации система проверяет роль пользователя и предоставляет доступ к административным функциям только администратору. Такой подход необходим для защиты данных о товарах, заказах, клиентах и платежах. Обычные пользователи не должны иметь возможности изменять ассортимент, просматривать данные других клиентов или управлять заказами.

Основным разделом административной панели является раздел управления товарами. В нем администратор может добавлять новые товары, редактировать существующие позиции и удалять товары, которые больше не должны отображаться в каталоге. При создании или редактировании товара указываются его название, описание, цена, категория, подкатегория, изображения и набор характеристик. Так как товары могут относиться к разным категориям, административная панель должна поддерживать гибкую работу с характеристиками. Например, для электроники могут задаваться бренд, объем памяти, процессор или емкость аккумулятора, а для одежды — размер, цвет, материал и другие параметры.

Важной частью управления товарами является работа с изображениями. Администратор должен иметь возможность загружать изображения товара, обновлять их или удалять при необходимости. После загрузки изображение передается через серверную часть во внешний сервис хранения, а в базе данных

сохраняется ссылка на него. Это позволяет использовать одни и те же изображения как в мобильном приложении, так и в административной панели.

Раздел управления заказами предназначен для просмотра и обработки заказов, оформленных пользователями через мобильное приложение. В этом разделе администратор может видеть список заказов, дату их создания, данные пользователя, состав заказа, итоговую стоимость, адрес доставки и текущий статус. Также должна быть предусмотрена возможность изменения статуса заказа, например: создан, оплачен, находится в обработке, отправлен, доставлен или отменен. Это позволяет администратору контролировать процесс выполнения заказа после его оплаты.

При проектировании раздела заказов важно учитывать, что заказ связан сразу с несколькими сущностями системы: пользователем, товарами, адресом доставки, платежом и статусом выполнения. Поэтому интерфейс административной панели должен показывать информацию в удобном и понятном виде. Администратор должен быстро находить нужный заказ, проверять его детали и при необходимости изменять статус. Для этого могут использоваться поиск, фильтрация по статусу, сортировка по дате и просмотр подробной информации о заказе.

Раздел клиентов предназначен для просмотра информации о пользователях системы. В нем администратор может видеть список зарегистрированных пользователей, их основные данные, количество заказов и историю покупок. Такая информация может использоваться для анализа активности клиентов, решения вопросов по заказам и оценки поведения покупателей. При этом административная панель не должна предоставлять доступ к лишним или чувствительным данным, если они не требуются для работы магазина.

Отдельное значение имеет раздел статистики. Он предназначен для отображения основных показателей работы интернет-магазина. В данном разделе могут отображаться общая выручка, количество заказов, количество зарегистрированных клиентов, число товаров в каталоге и другие сводные данные. Такая информация позволяет администратору быстро оценить текущее состояние магазина и понять, какие направления требуют внимания.

Статистический раздел не заменяет полноценную аналитическую систему, однако он помогает получить базовое представление о работе проекта. Например, администратор может увидеть, сколько заказов было оформлено, какие товары добавлены в каталог, сколько пользователей зарегистрировано и как изменяется общая активность магазина. В дальнейшем этот раздел может быть расширен за счет графиков, фильтров по датам и анализа популярных товаров.

Все действия в административной панели выполняются через серверную API-часть. Это означает, что панель не обращается напрямую к базе данных, а отправляет запросы на сервер. Сервер проверяет права администратора, валидирует

переданные данные и только после этого выполняет соответствующую операцию. Такой подход повышает безопасность системы и снижает риск некорректного изменения данных.

Интерфейс административной панели должен быть простым и понятным, так как он предназначен для регулярной работы с данными. Основные разделы должны быть доступны через навигационное меню, а формы добавления и редактирования товаров должны быть логично структурированы. При возникновении ошибок система должна показывать администратору понятные сообщения, например о незаполненных обязательных полях или некорректных значениях.

Административная панель является важной частью проектируемой системы, поскольку именно через нее выполняется основное управление интернет-магазином. Она объединяет работу с товарами, изображениями, заказами, клиентами и статистикой в одном интерфейсе. За счет этого администратор получает инструмент не только для наполнения каталога, но и для контроля всего процесса продажи: от добавления товара до обработки заказа и анализа результатов работы магазина.

Наличие отдельной административной панели делает систему более удобной для эксплуатации и дальнейшего развития. В будущем ее функциональность может быть расширена за счет более подробной аналитики, фильтрации заказов по периодам, управления скидками, промокодами, категориями товаров и настройками отображения каталога в мобильном приложении. Такой подход позволяет рассматривать административную панель не просто как вспомогательный интерфейс, а как полноценный инструмент управления электронной коммерцией.

2.5 Проектирование базы данных

База данных является основным хранилищем информации разрабатываемой системы электронной коммерции. В ней сохраняются данные о пользователях, товарах, корзинах, заказах, адресах доставки, отзывах и избранных товарах. Так как проект использует MongoDB, структура базы данных строится на коллекциях документов, что удобно для хранения объектов с различным набором полей и вложенных данных.

В проекте предусмотрены пять основных коллекций: users, products, carts, orders и reviews. Каждая из них отвечает за отдельную область данных и связана с другими коллекциями через идентификаторы документов. Такой подход позволяет разделить данные по смысловым сущностям и при этом сохранить связи между пользователями, товарами, заказами и отзывами.

Коллекция users предназначена для хранения информации о пользователях системы. В ней сохраняются электронная почта, имя, изображение профиля, идентификатор пользователя из Clerk, идентификатор клиента в Stripe, список адресов доставки и список избранных товаров. Адреса доставки хранятся как вложенные документы внутри пользователя, так как они относятся только к конкретному аккаунту и используются при оформлении заказа. Список избранного реализован через массив ссылок на товары, что позволяет пользователю сохранять интересные позиции и быстро возвращаться к ним в мобильном приложении.

Коллекция products используется для хранения товаров интернет-магазина. В ней содержатся название, описание, цена, количество товара на складе, категория, подкатегория, изображения, средний рейтинг и количество отзывов. Для характеристик товара используется поле specs, которое может содержать разные наборы параметров в зависимости от категории товара. Например, для электроники это могут быть бренд, память, процессор или емкость аккумулятора, а для одежды — размер, цвет и материал. Такой подход делает каталог более гибким и позволяет расширять ассортимент без полной переработки структуры базы данных.

Коллекция carts отвечает за корзину пользователя. Каждая корзина связана с конкретным пользователем и содержит массив товаров с указанием количества. Элементы корзины хранятся как вложенные документы, так как они существуют только в рамках корзины и не используются отдельно от нее. При изменении количества товара или удалении позиции обновляется соответствующий элемент корзины.

Коллекция orders предназначена для хранения заказов. Каждый заказ связан с пользователем и содержит список заказанных товаров, адрес доставки, результат оплаты, итоговую стоимость и статус заказа. Важной особенностью является то, что в заказе сохраняется не только ссылка на товар, но и снимок его названия, цены и изображения на момент покупки. Это необходимо для того, чтобы данные заказа оставались корректными даже в случае последующего изменения товара в каталоге.

Статус заказа ограничивается заданным набором значений: pending, shipped и delivered. Это позволяет контролировать процесс обработки заказа и отображать пользователю актуальное состояние покупки. Также в заказе могут сохраняться даты отправки и доставки, что необходимо для отслеживания выполнения заказа и последующего добавления отзыва.

Коллекция reviews используется для хранения пользовательских оценок товаров. Каждый отзыв связан с пользователем, товаром и заказом. Такая структура позволяет проверять, что отзыв оставлен пользователем, который действительно оформил заказ на данный товар. Оценка хранится в диапазоне от 1 до 5, а на основе отзывов в карточке товара может отображаться средний рейтинг и общее количество оценок.

Между основными коллекциями базы данных существуют следующие связи. Один пользователь имеет одну корзину и может иметь несколько заказов. Пользователь может добавлять несколько товаров в избранное, а один товар может находиться в избранном у разных пользователей. Корзина содержит несколько элементов, каждый из которых связан с конкретным товаром. Заказ также содержит несколько позиций, каждая из которых относится к определенному товару. Кроме того, пользователь может оставлять несколько отзывов, товар может иметь несколько отзывов, а каждый отзыв связан с конкретным заказом. Структура базы данных и основные связи между коллекциями представлены на рисунке 2.3.

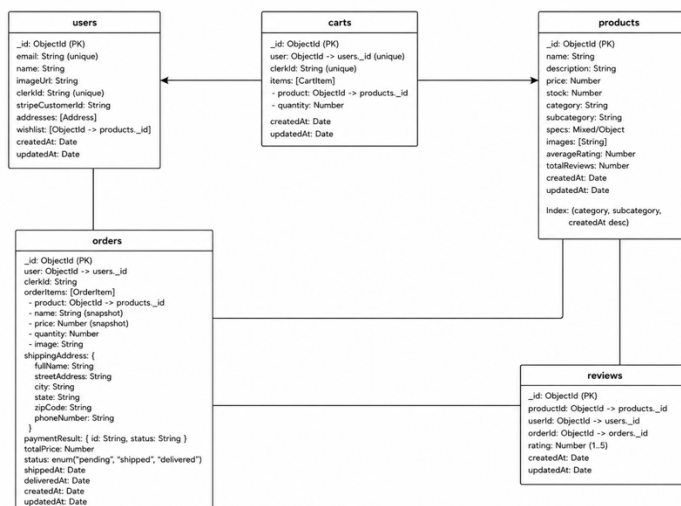


Рисунок 2.3 – Структура базы данных

Для ускорения работы с каталогом товаров предусматривается индекс по категории, подкатегории и дате создания товара. Это позволяет быстрее получать товары определенной категории и выводить их в нужном порядке. Такой индекс особенно важен для экрана каталога, где пользователь часто использует категории, подкатегории, фильтрацию и сортировку.

Проектируемая структура базы данных соответствует основным сценариям работы системы: регистрации пользователя, просмотру каталога, добавлению товаров в избранное и корзину, оформлению заказа, оплате, отслеживанию статуса заказа и добавлению отзывов. За счет использования ссылок между коллекциями и вложенных документов база данных остается достаточно простой, но при этом поддерживает все необходимые связи между сущностями проекта.

Выбор технологий для разработки системы выполнялся с учетом структуры проекта, его функциональных требований и необходимости обеспечить работу трех основных частей: мобильного приложения, серверной API-части и административной панели. Разрабатываемая система представляет собой платформу электронной коммерции, в которой пользователь работает с мобильным приложением, администратор управляет магазином через веб-панель, а серверная

часть связывает интерфейсы, базу данных, платежи, авторизацию и внешние сервисы в единую систему .

Для разработки мобильного приложения выбраны React Native и Expo. React Native позволяет создавать мобильные приложения на основе React и использовать единый подход к разработке интерфейса для разных мобильных платформ [4]. Expo применяется для упрощения разработки, запуска, тестирования и сборки мобильного приложения [8]. Такой набор технологий подходит для проекта, так как позволяет быстрее реализовать мобильный клиент интернет-магазина без необходимости отдельно разрабатывать нативные приложения для разных операционных систем.

Для организации навигации в мобильном приложении используется Expo Router. Он позволяет строить структуру экранов на основе файловой маршрутизации, что делает навигацию между каталогом, карточкой товара, корзиной, профилем и заказами более понятной для разработки и сопровождения [9]. Для загрузки и обновления данных применяется TanStack Query. Эта библиотека используется для работы с серверными данными, их кэширования и обновления, что особенно важно для каталога товаров, корзины, избранного и истории заказов [10].

Для разработки серверной части используются Node.js и Express. Express выбран как основной веб-фреймворк для создания API-маршрутов, обработки HTTP-запросов и взаимодействия мобильного приложения и административной панели с сервером [5]. Такой подход позволяет реализовать REST API для работы с пользователями, товарами, корзиной, заказами, оплатой, отзывами и административными функциями.

В качестве базы данных используется MongoDB. Она подходит для хранения данных в виде документов и коллекций, что удобно для системы электронной коммерции, где товары могут иметь разные категории, подкатегории и наборы характеристик [6]. Для работы с MongoDB применяется Mongoose, который позволяет описывать схемы данных, модели и правила валидации. Это упрощает обработку пользователей, товаров, корзин, заказов и отзывов на стороне серверной части.

Для авторизации пользователей используется Clerk. Данный сервис применяется как в мобильном приложении, так и в административной панели. Он позволяет организовать регистрацию, вход пользователей и проверку сессий. На серверной стороне Clerk используется для защиты маршрутов и разграничения доступа между обычными пользователями и администраторами.

Для реализации платежного функционала используется Stripe. В мобильном приложении Stripe применяется для отображения платежного интерфейса, а серверная часть отвечает за создание платежа и обработку результата оплаты. Подтверждение успешной оплаты выполняется через webhook-события, что

позволяет изменять статус заказа только после получения подтверждения от платежного сервиса [12]. Такой подход повышает надежность оформления заказов и снижает риск некорректной обработки платежей.

Для хранения изображений товаров используется Cloudinary. Администратор загружает изображения через административную панель, после чего серверная часть передает их во внешний сервис хранения. В базе данных сохраняются ссылки на изображения, которые затем используются в мобильном приложении и административной панели. Такой подход позволяет не хранить изображения непосредственно в базе данных и упростить работу с медиафайлами.

Административная панель разрабатывается с использованием React и Vite. React используется для создания интерфейса управления товарами, заказами, клиентами и статистикой, а Vite — для быстрой сборки и разработки веб-приложения. Для стилизации интерфейса применяются Tailwind CSS и DaisyUI, что позволяет быстрее создавать аккуратные и понятные элементы интерфейса.

Для модуля интеллектуального подбора товаров используется Google Gemini API. Данный сервис применяется для обработки пользовательских запросов в свободной форме и уточнения рекомендаций по товарам [7]. При этом система не зависит полностью от внешней модели: сначала сервер может выполнить локальный поиск и ранжирование товаров по каталогу, а затем использовать Gemini для улучшения результата. Это делает модуль более устойчивым и позволяет сохранить базовую работоспособность даже при недоступности внешнего AI-сервиса.

Для повышения надежности проекта также предусматривается использование Sentry и Inngest. Sentry применяется для мониторинга ошибок в мобильном приложении и административной панели [14]. Inngest может использоваться для обработки фоновых задач и событий, например связанных с пользователями, заказами или уведомлениями. Эти инструменты помогают улучшить сопровождение системы и упростить обработку асинхронных процессов.

В результате выбранный набор технологий позволяет реализовать все основные части проекта: мобильное приложение для покупателей, серверную API-часть, административную панель, базу данных, авторизацию, оплату, хранение изображений и интеллектуальный подбор товаров. Используемые технологии соответствуют структуре системы, в которой мобильный клиент, backend API и admin dashboard работают совместно, а внешние сервисы подключаются для выполнения специализированных задач.

3. РАЗРАБОТКА СИСТЕМЫ

3.1 Реализация функциональных возможностей мобильного приложения

Мобильное приложение является основной пользовательской частью разрабатываемой системы электронной коммерции. Через него покупатель получает доступ к каталогу товаров, просматривает подробную информацию о продукции, формирует корзину, оформляет заказ, выбирает адрес доставки, выполняет оплату и отслеживает состояние своих заказов. Основной задачей при разработке мобильного приложения являлась реализация полного пользовательского сценария покупки товара — от входа в систему до завершения заказа и дальнейшего взаимодействия с приобретённой продукцией.

Функциональность мобильного приложения была реализована с учётом типичных потребностей пользователя интернет-магазина. Приложение предоставляет возможность авторизации, просмотра ассортимента, поиска и фильтрации товаров, добавления товаров в избранное и корзину, оформления заказа, управления адресами доставки, просмотра истории заказов и добавления отзывов. За счёт этого мобильная часть системы обеспечивает основные операции, необходимые для полноценного использования электронной торговой платформы.

Одной из первых реализованных возможностей является авторизация пользователя. Для доступа к персональным функциям приложения, таким как корзина, избранное, оформление заказа и просмотр истории покупок, пользователь должен пройти вход в систему. Авторизация необходима для связывания действий пользователя с его учётной записью и корректного хранения персональных данных, заказов и адресов доставки. В приложении предусмотрена возможность входа через сторонние сервисы, что упрощает процесс регистрации и снижает количество действий, необходимых для начала работы с системой.

После авторизации пользователь получает доступ к каталогу товаров. Каталог является одной из центральных функциональных частей приложения, так как именно через него осуществляется первичный выбор продукции. В каталоге отображается список товаров, загружаемых с серверной части системы. Каждый товар содержит основную информацию, необходимую для принятия решения о дальнейшем просмотре: название, цену, изображение, категорию и краткие характеристики. Данные о товарах не хранятся статически внутри мобильного приложения, а запрашиваются с backend API, что позволяет администраторам изменять ассортимент без необходимости обновления клиентского приложения.

Для повышения удобства работы с каталогом была реализована возможность поиска и фильтрации товаров. Пользователь может находить товары по названию, категории или другим параметрам, связанным с характеристиками продукции. Это особенно важно для электронной торговой платформы, так как при большом количестве товаров ручной просмотр всего ассортимента становится неудобным. Фильтрация позволяет сократить количество отображаемых товаров и быстрее перейти к подходящим вариантам. Наличие данной функции делает взаимодействие с каталогом более удобным и помогает пользователю быстрее находить нужную продукцию.

При выборе конкретного товара пользователь переходит к его подробному описанию. Карточка товара содержит расширенную информацию: изображение, название, цену, описание, категорию, подкатеорию, характеристики, рейтинг и доступность. Данный экран выполняет функцию детального ознакомления с товаром перед покупкой. Пользователь может оценить параметры товара, сравнить их со своими потребностями и принять решение о добавлении товара в корзину или избранное. Наличие характеристик особенно важно для товаров разных категорий, поскольку для электроники, одежды, спортивных товаров и книг используются разные наборы параметров.

В приложении реализована функция добавления товаров в избранное. Она позволяет пользователю сохранить интересующие товары без необходимости сразу оформлять заказ. Избранное используется как персональный список отложенных товаров, к которому можно вернуться позже. Эта функция повышает удобство работы с приложением, так как пользователь может сравнивать товары, откладывать покупку или сохранять позиции, которые заинтересовали его в процессе просмотра каталога. Данные об избранных товарах связаны с конкретным пользователем и сохраняются в базе данных.

Одной из ключевых функций мобильного приложения является корзина. Корзина предназначена для временного хранения товаров, которые пользователь планирует приобрести. При добавлении товара в корзину сохраняется информация о выбранной позиции, количестве и стоимости. Пользователь может изменять количество товаров, удалять отдельные позиции или полностью очищать корзину. Итоговая стоимость заказа рассчитывается на основе актуальных данных о товарах. При этом окончательная проверка цены и доступности выполняется на серверной стороне, что позволяет избежать ошибок и некорректного оформления заказа при изменении данных о товаре.

После формирования корзины пользователь может перейти к оформлению заказа. На данном этапе он выбирает адрес доставки, проверяет состав заказа и подтверждает намерение выполнить покупку. Для корректного оформления заказа в приложении предусмотрена работа с адресами пользователя. Пользователь может

добавлять, редактировать и выбирать адрес доставки. Адреса сохраняются в профиле пользователя и могут использоваться повторно при последующих заказах. Это упрощает процесс покупки и делает оформление заказа более удобным.

Оплата заказа реализуется через интеграцию с платёжным сервисом. Мобильное приложение инициирует процесс оплаты, однако критически важные операции, связанные с созданием платёжной сессии, проверкой суммы и подтверждением заказа, выполняются на серверной стороне. Такой подход позволяет повысить безопасность системы, так как клиентское приложение не отвечает за окончательное подтверждение оплаты и не может самостоятельно изменить стоимость заказа. После успешной оплаты информация передаётся на сервер, где заказ создаётся или обновляется в соответствии с подтверждённым платёжным событием.

Важной частью функциональности является работа с заказами. После оформления покупки пользователь может просматривать список своих заказов и отслеживать их состояние. Для каждого заказа отображается информация о составе, стоимости, адресе доставки и текущем статусе. Статусы позволяют пользователю понимать, на каком этапе находится заказ: создан, оплачен, обрабатывается, доставляется или завершён. Такая логика формирует полноценный post-purchase сценарий, при котором взаимодействие пользователя с системой не заканчивается сразу после оплаты.

После завершения заказа пользователь получает возможность оставить отзыв и оценку на приобретённый товар. Эта функция важна как для самого пользователя, так и для системы в целом. Отзывы помогают другим покупателям принимать решение о покупке, а рейтинг товара может использоваться при сортировке и отображении продукции в каталоге. Возможность оставить отзыв ограничивается товарами, которые действительно были приобретены и доставлены пользователю. Это позволяет снизить вероятность появления недостоверных оценок и повысить качество пользовательского контента.

Также в мобильном приложении реализован профиль пользователя. В профиле сосредоточены персональные функции: просмотр данных учётной записи, управление адресами, доступ к заказам и избранным товарам. Профиль обеспечивает пользователю удобную точку входа к информации, связанной с его действиями в системе. Благодаря этому пользователь может не только совершать покупки, но и управлять своими данными и историей взаимодействия с платформой.

Отдельное место в функциональности мобильного приложения занимает интеллектуальный подбор товаров. Пользователь может сформулировать запрос в свободной форме, например описать желаемый товар, назначение покупки или собственные предпочтения. После этого приложение передаёт запрос на серверную часть, где выполняется обработка и подбор релевантных товаров. В мобильном

приложении результат отображается в виде ответа ассистента и списка подходящих товарных карточек. Подробная реализация данного модуля рассматривается в отдельной подглаве, однако с точки зрения мобильного приложения он является дополнительным способом поиска и выбора товаров.

Взаимодействие мобильного приложения с серверной частью построено через API-запросы. Приложение получает данные о товарах, заказах, корзине, адресах и пользователе с backend-сервера. Для работы с серверными данными используется механизм загрузки, кэширования и обновления информации. Это позволяет уменьшить количество повторных запросов, ускорить отображение данных и обеспечить более стабильную работу приложения. При изменении данных, например добавлении товара в корзину или оформлении заказа, приложение отправляет соответствующий запрос на сервер, после чего обновляет локальное состояние интерфейса.

При реализации функциональности учитывалась необходимость обработки ошибок и нестандартных ситуаций. Например, пользователь может попытаться оформить заказ с пустой корзиной, выбрать недоступный товар, выполнить действие без авторизации или столкнуться с ошибкой оплаты. В таких случаях приложение должно корректно реагировать на ситуацию и не допускать нарушения пользовательского сценария. Обработка ошибок позволяет повысить надёжность приложения и сделать взаимодействие с системой более предсказуемым.

В рамках мобильного приложения были реализованы основные функциональные возможности, необходимые для работы электронной торговой платформы. Пользователь может пройти авторизацию, просматривать каталог, искать и фильтровать товары, добавлять их в избранное и корзину, оформлять заказ, оплачивать покупку, управлять адресами доставки, просматривать историю заказов и оставлять отзывы. Реализованный функционал обеспечивает полный цикл взаимодействия покупателя с системой и создаёт основу для дальнейшего расширения приложения, включая развитие интеллектуального подбора товаров и персонализированных рекомендаций.

3.2 Реализация пользовательского интерфейса и навигации

Пользовательский интерфейс мобильного приложения был реализован с учётом основных сценариев взаимодействия покупателя с электронной торговой платформой. Главной задачей интерфейса являлось обеспечение удобного доступа к каталогу товаров, карточкам продукции, корзине, избранному, профилю пользователя и истории заказов. При разработке интерфейса учитывалась

необходимость простой навигации, понятного расположения элементов и минимального количества действий для выполнения основных операций.

Структура мобильного приложения построена по принципу разделения функциональных экранов. Пользователь может последовательно переходить между основными разделами приложения, не теряя контекста работы с системой. Основными экранами являются экран каталога товаров, экран подробной информации о товаре, экран корзины, экран оформления заказа, экран избранного, экран профиля и экран заказов. Такое разделение позволяет логично организовать пользовательский сценарий и сделать приложение более понятным для конечного пользователя.

Главным экраном приложения является каталог товаров (рис 3.1). На нём пользователь видит список доступной продукции, представленную в виде карточек. Каждая карточка содержит изображение товара, название, цену и краткую информацию, достаточную для первичного выбора. Из каталога пользователь может перейти к подробному просмотру товара, добавить позицию в избранное или продолжить поиск нужной продукции. Такой подход делает каталог центральной точкой взаимодействия пользователя с системой.

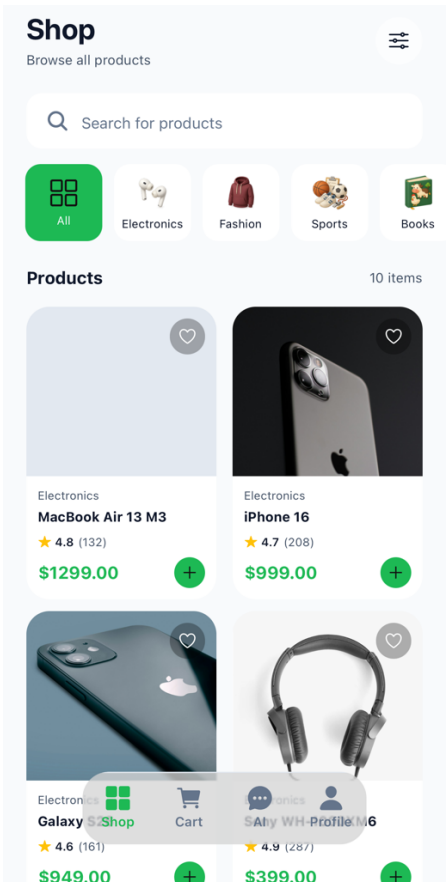


Рисунок 3.1 – Экран каталога товаров

Для удобства пользователя интерфейс каталога предусматривает визуальное разделение товаров и акцент на наиболее важной информации. Изображение товара

помогает быстрее ориентироваться в ассортименте, а цена и название позволяют оценить товар без перехода на дополнительный экран. При большом количестве товаров такой формат отображения упрощает просмотр каталога и делает взаимодействие с приложением более быстрым.

После выбора товара пользователь переходит на экран карточки товара (рис 3.2). На данном экране отображается расширенная информация о продукции: изображение, название, стоимость, описание, категория, характеристики и рейтинг. Карточка товара предназначена для более детального ознакомления с выбранной позицией перед добавлением её в корзину или избранное. Здесь же размещаются основные элементы управления, позволяющие пользователю выполнить дальнейшие действия с товаром.

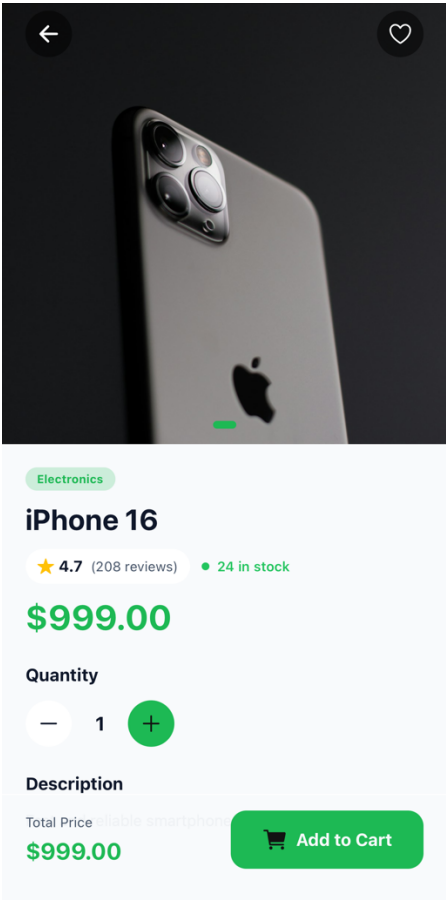


Рисунок 3.2 – Экран карточки товара

Экран карточки товара реализован таким образом, чтобы пользователь мог быстро получить всю необходимую информацию о выбранной позиции. Основной акцент сделан на изображении товара, его названии, цене и характеристиках. Кнопки добавления в корзину и избранное располагаются в доступной зоне, что упрощает выполнение целевых действий. Такое расположение элементов снижает количество лишних переходов и делает процесс выбора товара более удобным.

Навигация внутри мобильного приложения организована таким образом, чтобы пользователь мог быстро переходить между ключевыми разделами. Для

основных разделов используется нижняя навигация, позволяющая перейти к каталогу, корзине, избранному и профилю. Такой способ навигации является привычным для мобильных приложений и обеспечивает быстрый доступ к часто используемым функциям. Внутри отдельных сценариев, например при просмотре товара или оформлении заказа, используются последовательные переходы между экранами.

Отдельное внимание было уделено экрану корзины. Корзина отображает список товаров, которые пользователь планирует приобрести. Для каждой позиции выводится название, изображение, цена, количество и итоговая стоимость. Пользователь может изменить количество товара или удалить его из корзины. После проверки состава корзины пользователь может перейти к оформлению заказа. Данный экран является важной частью пользовательского сценария, так как связывает процесс выбора товаров с дальнейшей оплатой.

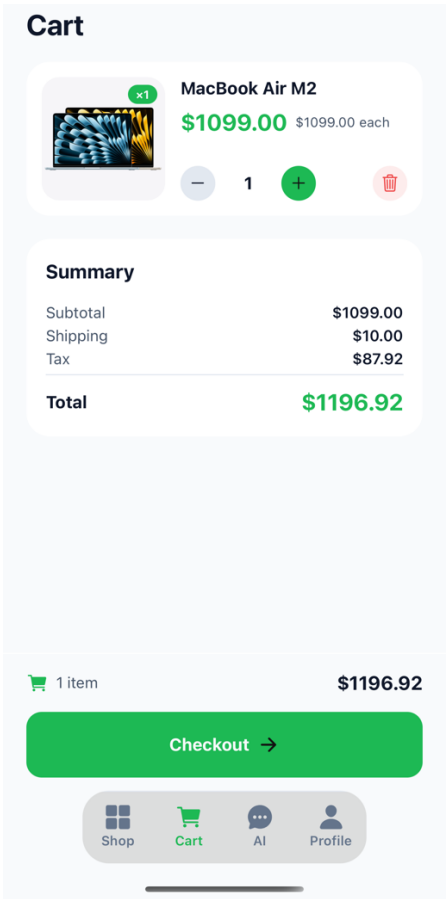


Рисунок 3.3 – Экран корзины

Интерфейс корзины был реализован с акцентом на понятность состава заказа. Пользователь видит выбранные товары и может проверить итоговую стоимость до перехода к оплате. Это снижает вероятность ошибки при оформлении заказа и позволяет пользователю контролировать свои действия. Переход к

оформлению заказа выполняется через отдельную кнопку, что визуально отделяет этап просмотра корзины от этапа подтверждения покупки.

Экран оформления заказа продолжает сценарий покупки и позволяет пользователю выбрать адрес доставки, проверить информацию о заказе и перейти к оплате. В интерфейсе данного экрана важно обеспечить последовательность действий: сначала пользователь проверяет данные, затем подтверждает заказ и только после этого выполняет оплату. Такая структура делает процесс оформления заказа более предсказуемым и снижает вероятность случайных действий.

Профиль пользователя объединяет персональные функции приложения. Через него пользователь может получить доступ к своим заказам, адресам доставки, избранным товарам и данным учётной записи. Размещение этих возможностей в одном разделе делает управление личной информацией более удобным. Пользователь может быстро найти нужную информацию без необходимости переходить через каталог или корзину.

Экран заказов предназначен для отображения истории покупок и текущих статусов заказов. В нём пользователь может просмотреть информацию о ранее оформленных заказах, их составе, стоимости и состоянии выполнения. Наличие статусов позволяет пользователю понимать, на каком этапе находится заказ. Это особенно важно для электронной коммерции, так как пользователь ожидает прозрачности после оформления покупки.

В интерфейсе также предусмотрены состояния загрузки, пустых списков и ошибок. Например, если каталог товаров ещё загружается, пользователь видит соответствующее состояние ожидания. Если корзина или избранное пусты, приложение отображает понятное сообщение вместо пустого экрана. При возникновении ошибки загрузки данных пользователь получает уведомление о проблеме. Такие состояния необходимы для повышения удобства использования приложения и обеспечения предсказуемого поведения интерфейса.

Визуальное оформление мобильного приложения выполнено в едином стиле. Для экранов используются одинаковые принципы расположения элементов, отступов, кнопок, карточек и текстовой информации. Единообразие интерфейса помогает пользователю быстрее привыкнуть к приложению и понимать, как выполнять необходимые действия. Карточки товаров, кнопки действий и навигационные элементы имеют повторяющуюся структуру, что делает интерфейс более целостным.

Реализованная навигация обеспечивает полный пользовательский путь от просмотра каталога до оформления заказа. Пользователь может открыть каталог, выбрать товар, изучить его характеристики, добавить товар в корзину, перейти к оформлению заказа, выбрать адрес доставки и выполнить оплату. После завершения покупки он может вернуться к списку заказов и отслеживать их состояние. Такая

структура интерфейса соответствует основным требованиям мобильного приложения электронной коммерции и обеспечивает удобное взаимодействие пользователя с системой.

3.3 Реализация серверной части

Серверная часть является центральным компонентом разрабатываемой системы электронной коммерции, так как именно она обеспечивает взаимодействие мобильного приложения, административной панели, базы данных и внешних сервисов. Backend отвечает за обработку запросов пользователей, выполнение бизнес-логики, проверку прав доступа, работу с товарами, корзиной, заказами, оплатой, адресами доставки, отзывами и модулем интеллектуального подбора товаров.

Серверная часть была реализована на основе Node.js и фреймворка Express. Такой подход позволяет организовать REST API, через которое мобильное приложение и административная панель получают доступ к данным системы. Сервер принимает HTTP-запросы от клиентских приложений, обрабатывает их, обращается к базе данных и возвращает результат в формате JSON. За счёт этого обеспечивается разделение клиентской и серверной логики: мобильное приложение отвечает за отображение интерфейса, а backend — за хранение данных, безопасность и выполнение основных операций.

Архитектура серверной части построена по модульному принципу. Основная функциональность разделена на отдельные группы маршрутов, связанные с пользователями, товарами, корзиной, заказами, адресами, отзывами, оплатой и интеллектуальным подбором товаров. Такой подход упрощает поддержку проекта, так как каждая часть системы отвечает за свой набор операций. Например, маршруты товаров используются для получения каталога и подробной информации о товаре, маршруты корзины — для добавления и удаления товаров, а маршруты заказов — для оформления и отслеживания покупок.

Одним из ключевых направлений реализации серверной части является работа с пользователями. После авторизации пользователя в мобильном приложении сервер получает данные об учётной записи и связывает действия пользователя с соответствующей записью в базе данных. Это необходимо для хранения корзины, избранных товаров, адресов доставки и истории заказов. Защищённые маршруты backend API доступны только авторизованным пользователям, что позволяет предотвратить несанкционированный доступ к персональным данным.

Для защиты маршрутов используется middleware авторизации. Он проверяет данные пользователя перед выполнением запроса и определяет, имеет ли пользователь право обращаться к конкретному ресурсу. Например, пользователь может просматривать только свои заказы и адреса доставки, а административные функции доступны только пользователям с соответствующей ролью. Такой механизм разграничения доступа является обязательным для системы, в которой присутствуют как обычные покупатели, так и администраторы.

Работа с товарами реализована через отдельный набор API-маршрутов. Сервер предоставляет возможность получать список товаров, выполнять поиск, фильтрацию по категориям и подкатегориям, а также получать подробную информацию о конкретном товаре. Данные о товарах хранятся в базе данных и включают название, описание, цену, изображения, категорию, подкатеорию, характеристики, рейтинг и информацию о наличии. Благодаря этому мобильное приложение может динамически отображать актуальный каталог без необходимости хранить данные локально.

Особое значение имеет обработка характеристик товаров. В системе предусмотрена возможность хранения разных наборов характеристик для разных категорий и подкатегорий. Например, для электроники могут использоваться параметры, связанные с брендом, процессором, памятью или ёмкостью аккумулятора, а для одежды — размер, цвет и материал. Такая структура делает каталог более гибким и позволяет масштабировать систему при добавлении новых типов товаров.

Корзина пользователя также обрабатывается на серверной стороне. При добавлении товара в корзину backend проверяет существование товара, его доступность и корректность переданных данных. Пользователь может изменять количество товаров, удалять позиции и получать актуальное содержимое корзины. Серверная обработка корзины позволяет хранить её состояние независимо от устройства пользователя и обеспечивает доступ к актуальным данным после повторной авторизации.

Оформление заказа реализовано как отдельный серверный сценарий. При создании заказа сервер получает данные о товарах из корзины, адрес доставки и информацию о пользователе. Перед переходом к оплате backend выполняет проверку актуальной стоимости товаров и их доступности. Это важно, поскольку данные на клиентской стороне не должны считаться окончательными: цена товара или его наличие могут измениться после того, как пользователь добавил его в корзину. Окончательное подтверждение заказа выполняется только после успешной оплаты.

Платёжная логика реализована через интеграцию со Stripe. Мобильное приложение инициирует процесс оплаты, однако создание платёжной сессии и

проверка суммы выполняются на сервере. Такой подход повышает безопасность системы, так как пользователь не может самостоятельно изменить итоговую стоимость заказа на стороне клиента. После завершения оплаты Stripe отправляет событие на webhook backend-сервера. Сервер обрабатывает это событие и обновляет состояние заказа в базе данных.

Webhook-обработка является важной частью платёжного сценария. Она позволяет серверу получать подтверждение оплаты напрямую от платёжного сервиса, а не полагаться только на ответ мобильного приложения. Это снижает риск ошибок при нестабильном соединении или преждевременном закрытии приложения пользователем. После получения подтверждённого события сервер создаёт или обновляет заказ, очищает корзину пользователя и фиксирует факт успешной оплаты.

Кроме пользовательских функций, серверная часть обслуживает административную панель. Для администратора реализованы API-маршруты управления товарами, заказами, клиентами и статистикой. Администратор может создавать, редактировать и удалять товары, изменять их характеристики, загружать изображения, просматривать заказы и управлять их статусами. Также сервер предоставляет данные для отображения статистики: количество заказов, товаров, клиентов и общую выручку. Доступ к этим маршрутам ограничен и требует административных прав.

Для хранения изображений товаров используется внешний сервис Cloudinary. Серверная часть отвечает за передачу изображений, получение ссылок на загруженные файлы и сохранение этих ссылок в базе данных. Такой подход позволяет не хранить изображения непосредственно на сервере или в базе данных, а использовать специализированное облачное хранилище. В результате уменьшается нагрузка на backend и упрощается работа с медиафайлами.

Отдельное место в серверной части занимает модуль интеллектуального подбора товаров. Его задача заключается в обработке пользовательского запроса, сформулированного в свободной форме, и подборе подходящих товаров из каталога. В отличие от классического поиска, где пользователь должен указать точное название товара или выбрать фильтры, интеллектуальный подбор позволяет описать потребность обычным языком. Например, пользователь может написать, что ему нужна игровая мышь, удобные наушники для учёбы или недорогой смартфон с хорошей батареей (рис 3.4).

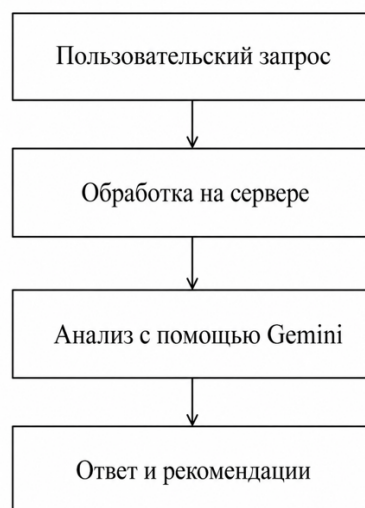


Рисунок 3.4 – Схема работы модуля интеллектуального подбора товаров

Модуль интеллектуального подбора реализован на стороне backend, так как именно сервер имеет доступ к базе данных товаров и может безопасно взаимодействовать с внешним AI-сервисом. Мобильное приложение отправляет текстовый запрос пользователя на соответствующий API-маршрут. После этого сервер выполняет обработку запроса, анализирует доступные товары и возвращает клиенту список наиболее подходящих вариантов вместе с текстовым ответом ассистента.

Обработка запроса в модуле интеллектуального подбора выполняется в несколько этапов. Сначала сервер получает текстовое описание потребности пользователя. Затем выполняется предварительный поиск по каталогу товаров. На этом этапе могут учитываться название товара, категория, подкатегория, описание и характеристики. Такая предварительная фильтрация позволяет сократить количество товаров, которые необходимо анализировать дальше, и повысить релевантность результата.

После предварительного отбора товаров сервер формирует контекст для AI-модели. В этот контекст передаётся запрос пользователя и информация о найденных товарах. Далее выполняется обращение к Gemini API, который анализирует пользовательский запрос и помогает определить, какие товары лучше соответствуют описанным потребностям. Использование языковой модели позволяет учитывать не только точные совпадения по словам, но и общий смысл запроса.

Например, если пользователь пишет, что ему нужна «мышка для игр», система может определить, что речь идёт о товаре из категории электроники, связанной с компьютерными аксессуарами. При этом пользователь может не указывать точное название модели, бренд или технические параметры. Модуль

интеллектуального подбора помогает связать такой естественный запрос с конкретными товарами из каталога.

Результат работы AI-модуля возвращается в мобильное приложение в структурированном виде. Ответ может включать текстовое пояснение и список рекомендованных товаров. Каждый товар содержит данные, необходимые для отображения карточки в интерфейсе: название, цену, изображение, категорию и идентификатор. Благодаря этому пользователь может сразу перейти от рекомендации к карточке товара и продолжить стандартный сценарий покупки.

Важной особенностью реализации является использование гибридного подхода. Система не передаёт всю ответственность за подбор товаров языковой модели. Сначала backend выполняет локальный поиск и предварительное ранжирование по товарам из базы данных, а затем AI-модель используется для улучшения качества рекомендации и формирования более понятного ответа. Такой подход позволяет сохранить контроль над данными, уменьшить объём передаваемой информации и повысить стабильность работы модуля.

Для повышения надёжности предусмотрен fallback-механизм. Если внешний AI-сервис временно недоступен или возвращает ошибку, система может использовать результаты локального поиска без участия языковой модели. В этом случае пользователь всё равно получает список подходящих товаров, а работа приложения не блокируется полностью. Такой подход важен для пользовательского опыта, поскольку недоступность внешнего сервиса не должна приводить к полной неработоспособности поиска.

Серверная реализация интеллектуального подбора также позволяет скрыть ключи доступа к AI-сервису от клиентского приложения. Мобильное приложение не обращается к Gemini API напрямую, а отправляет запрос только на backend. Это повышает безопасность системы, так как секретные ключи и логика формирования запроса находятся на серверной стороне. Кроме того, backend может контролировать количество запросов, обрабатывать ошибки и ограничивать передаваемые данные.

При разработке серверной части учитывалась необходимость обработки ошибок. Сервер должен корректно реагировать на ситуации, когда пользователь отправляет пустой запрос, товар не найден, заказ невозможно оформить, оплата не подтверждена или отсутствуют права доступа. Для этого в API используются проверки входных данных и возврат понятных сообщений об ошибках. Это позволяет клиентскому приложению корректно отображать состояние ошибки и не нарушать пользовательский сценарий.

Для взаимодействия с базой данных используется Mongoose. Он позволяет описывать схемы данных, выполнять запросы к MongoDB и управлять связями между сущностями. Через модели реализуется работа с пользователями, товарами, заказами, корзиной, отзывами и другими данными системы. Использование

Mongoose упрощает валидацию структуры документов и делает работу с базой данных более удобной на уровне серверного кода.

Серверная часть также обеспечивает связь между разными компонентами системы. Мобильное приложение использует backend для получения данных и выполнения пользовательских операций. Административная панель обращается к серверу для управления товарами, заказами и клиентами. Внешние сервисы, такие как Stripe, Cloudinary и Gemini, подключаются через backend, что позволяет централизованно контролировать бизнес-логику и безопасность приложения.

В рамках серверной части были реализованы API-маршруты для основных процессов электронной торговой платформы: авторизации пользователя, работы с каталогом товаров, корзиной, избранным, адресами доставки, заказами, отзывами, оплатой и административным управлением. Дополнительно был реализован модуль интеллектуального подбора товаров, который расширяет стандартные возможности поиска и позволяет пользователю находить товары с помощью запроса на естественном языке. Такая реализация backend обеспечивает работу мобильного приложения, административной панели и AI-функциональности в рамках единой системы.

3.4 Реализация базы данных

База данных является одним из ключевых компонентов разрабатываемой системы электронной коммерции, так как она обеспечивает хранение информации о пользователях, товарах, заказах, адресах доставки, избранных товарах, отзывах и платёжных данных. В рамках проекта база данных используется как единый источник данных для мобильного приложения, административной панели и серверной части. Все основные операции, выполняемые пользователем или администратором, связаны с чтением, созданием, изменением или удалением данных в базе.

Для хранения данных была использована документоориентированная база данных MongoDB. Согласно официальной документации, MongoDB хранит данные в виде документов и коллекций, что удобно для систем, где структура данных может отличаться в зависимости от типа сущности [6]. Такой подход подходит для каталога товаров, поскольку разные категории продукции могут иметь разные наборы характеристик.

Работа с базой данных в серверной части реализована с помощью Mongoose. Данная библиотека позволяет описывать схемы документов, задавать типы полей, обязательные значения и ограничения. Это даёт возможность сохранить гибкость

MongoDB, но при этом обеспечить более строгую структуру данных на уровне backend-приложения.

Подключение к базе данных выполняется при запуске серверной части. Строка подключения хранится в переменных окружения, что позволяет не размещать конфиденциальные параметры непосредственно в исходном коде. После успешного подключения backend получает доступ к коллекциям базы данных и выполняет операции чтения, создания, изменения и удаления данных через API-запросы. Серверная часть реализована на Express, который используется для обработки HTTP-запросов и построения API для мобильного приложения и административной панели [5].

Структура базы данных была разработана с учётом основных сущностей системы. К ним относятся пользователи, товары, заказы, корзина, отзывы и вложенные данные, например адреса доставки. Часть данных хранится в отдельных коллекциях, а часть — в виде вложенных документов. Такой подход позволяет упростить получение часто используемой информации и уменьшить количество дополнительных запросов к базе данных.

Одной из основных коллекций является коллекция пользователей. В ней хранятся адрес электронной почты, имя, изображение профиля, идентификатор пользователя во внешней системе авторизации, а также данные, необходимые для связи с платёжным сервисом. Для сопоставления пользователя с системой авторизации используется уникальный идентификатор Clerk. Это позволяет backend-серверу определять текущего пользователя и выполнять операции только с его личными данными.

Адреса доставки хранятся внутри документа пользователя в виде вложенных документов. Каждый адрес содержит данные, необходимые для оформления заказа: имя получателя, название адреса, улицу, город, почтовый индекс и страну. Такое решение удобно, так как адреса напрямую связаны с конкретным пользователем и часто используются вместе с его профилем или при оформлении покупки.

Для хранения избранных товаров используется связь пользователя с документами товаров. Вместо дублирования полной информации о товаре в документе пользователя сохраняется ссылка на соответствующий товар. Это позволяет всегда получать актуальные данные о цене, изображении и наличии товара из основной коллекции товаров.

Коллекция товаров является центральной для работы каталога. В ней хранятся название, описание, цена, изображения, категория, подкатегория, характеристики, рейтинг и информация о наличии. Эти данные используются мобильным приложением при отображении каталога, карточки товара, поиске, фильтрации и интеллектуальном подборе товаров.

Особое значение имеет хранение характеристик товаров. Поскольку в системе предусмотрены разные категории и подкатегории, структура характеристик должна быть гибкой. Например, для электроники могут использоваться параметры бренда, памяти, процессора или ёмкости аккумулятора, а для одежды — размер, цвет и материал. Использование документной модели позволяет хранить такие данные без необходимости создавать отдельную жёсткую таблицу для каждого типа товара.

Изображения товаров не сохраняются в базе данных в виде файлов. Вместо этого в документе товара хранятся ссылки на изображения, загруженные во внешний сервис хранения. Это уменьшает размер документов в MongoDB и упрощает работу с медиафайлами. Административная панель передаёт изображения на сервер, после чего backend сохраняет в базе только ссылки на загруженные файлы.

Для оформления покупок используется коллекция заказов. Заказ содержит идентификатор пользователя, список приобретённых товаров, количество каждой позиции, итоговую стоимость, адрес доставки, статус заказа и данные, связанные с оплатой. Состав заказа фиксируется на момент покупки, так как в дальнейшем цена или характеристики товара в каталоге могут измениться.

Статус заказа используется для отображения текущего этапа обработки покупки. Он может показывать, что заказ создан, оплачен, находится в обработке, доставляется или завершён. Эти данные отображаются как в мобильном приложении для пользователя, так и в административной панели для управления заказами.

Платёжные данные хранятся в базе данных в ограниченном виде. Система не сохраняет номера банковских карт или другую чувствительную информацию. Вместо этого используются идентификаторы, связанные с платёжным сервисом, например идентификатор клиента Stripe или платёжной операции. Такой подход повышает безопасность и позволяет связать заказ в системе с подтверждённой оплатой.

Корзина используется для временного хранения товаров, которые пользователь планирует приобрести. Для каждой позиции сохраняется ссылка на товар и количество. Перед оформлением заказа сервер повторно проверяет актуальную цену и наличие товаров, чтобы избежать несоответствий между данными на клиенте и реальным состоянием базы данных.

Отзывы пользователей связаны с товарами и заказами. Отзыв содержит оценку, текст комментария, идентификатор пользователя и ссылку на товар. Возможность оставить отзыв ограничивается товарами, которые были приобретены пользователем. Такая проверка выполняется на серверной стороне и позволяет повысить достоверность пользовательских оценок.

База данных также используется при работе модуля интеллектуального подбора товаров. При получении запроса от пользователя сервер обращается к коллекции товаров и выполняет поиск по названию, описанию, категории,

подкатегории и характеристикам. После этого отобранные данные могут быть переданы в AI-модуль для формирования рекомендаций. Интеграция с Gemini API используется для обработки естественного языка и уточнения релевантности подобранных товаров [7].

Для обеспечения корректности данных в схемах Mongoose используются обязательные поля, типы данных и ограничения уникальности. Например, электронная почта пользователя и идентификатор Clerk должны быть уникальными. Для товаров и заказов задаются обязательные поля, без которых запись не может быть создана корректно. Это снижает вероятность появления неполных или ошибочных данных.

Связи между сущностями реализуются с помощью идентификаторов документов. Пользователь связывается с заказами, адресами, избранными товарами и отзывами. Заказ связывается с пользователем и товарами, входящими в его состав. Такой подход позволяет сохранить логическую структуру данных и при необходимости получать связанную информацию через серверную часть.

В рамках реализации базы данных были созданы основные модели, необходимые для работы электронной торговой платформы. Они обеспечивают хранение пользователей, товаров, заказов, корзины, адресов доставки, избранных товаров, отзывов и данных, связанных с оплатой. Использование MongoDB и Mongoose позволило реализовать гибкую структуру данных, подходящую для каталога с разными категориями товаров, пользовательских сценариев покупки и дальнейшего расширения функциональности системы.

3.5 Реализация административной панели

Административная панель была реализована как отдельная веб-часть системы, предназначенная для управления электронной торговой платформой. В отличие от мобильного приложения, которое используется покупателями, административная панель предназначена для администратора системы. С её помощью выполняется управление товарами, заказами, клиентами и просмотр основных статистических показателей.

Административная панель играет важную роль в работе приложения, так как через неё наполняется каталог товаров и контролируется состояние заказов. Пользователь мобильного приложения видит только готовый результат: список товаров, карточки продукции, цены, изображения и статусы заказов. Администратор, в свою очередь, получает инструменты для создания, изменения и удаления данных без прямого обращения к базе данных.

Для реализации административной панели использовались React и Vite. Данные загружаются с серверной части через API-запросы. Для работы с серверными данными применяется TanStack Query, который позволяет получать, обновлять и кэшировать информацию. Визуальная часть панели реализована с использованием Tailwind-подхода и готовых UI-компонентов. Авторизация администратора выполняется через Clerk, а доступ к защищённым разделам контролируется на стороне backend API.

Доступ к административной панели ограничен, поскольку через неё можно изменять данные, влияющие на работу всей системы. После входа пользователь должен иметь соответствующие права администратора. Даже если обычный пользователь попытается обратиться к административным API-маршрутам напрямую, серверная часть должна заблокировать выполнение таких операций. Это позволяет защитить товары, заказы и данные клиентов от несанкционированного изменения.

Главный экран административной панели представляет собой информационную панель со статистикой. На нём отображаются основные показатели работы системы: количество товаров, число заказов, количество клиентов и общая выручка. Эти данные позволяют администратору быстро оценить состояние платформы и перейти к нужному разделу для дальнейшей работы (рис 3.5).

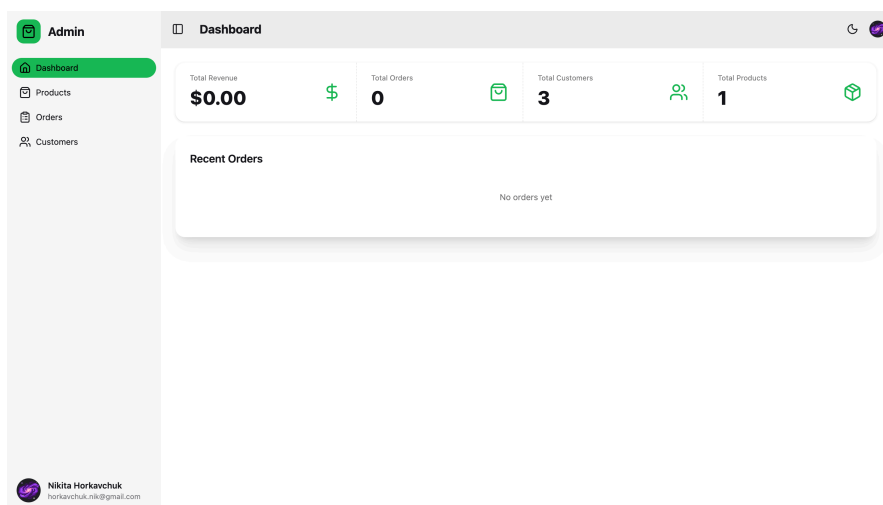


Рисунок 3.5 –Главный экран административной панели

Одним из основных разделов административной панели является управление товарами. В этом разделе администратор может просматривать список товаров, добавлять новые позиции, редактировать существующие товары и удалять их из каталога. Для каждого товара указываются название, описание, цена, категория, подкатегория, характеристики, наличие и изображения. После сохранения эти данные становятся доступными в мобильном приложении.

Особое внимание уделяется характеристикам товаров. Поскольку в системе используются разные категории и подкатегории, товары могут иметь различные наборы параметров. Например, для электроники важны технические характеристики, а для одежды — размер, цвет и материал. Возможность редактирования характеристик через административную панель делает каталог более гибким и удобным для дальнейшего расширения.

Изображения товаров также добавляются через административную панель. Администратор загружает файл, после чего серверная часть передаёт изображение во внешний сервис хранения и сохраняет в базе данных ссылку на него. Такой подход позволяет не хранить изображения непосредственно в базе данных и упрощает их дальнейшее отображение в мобильном приложении (рис 3.6).

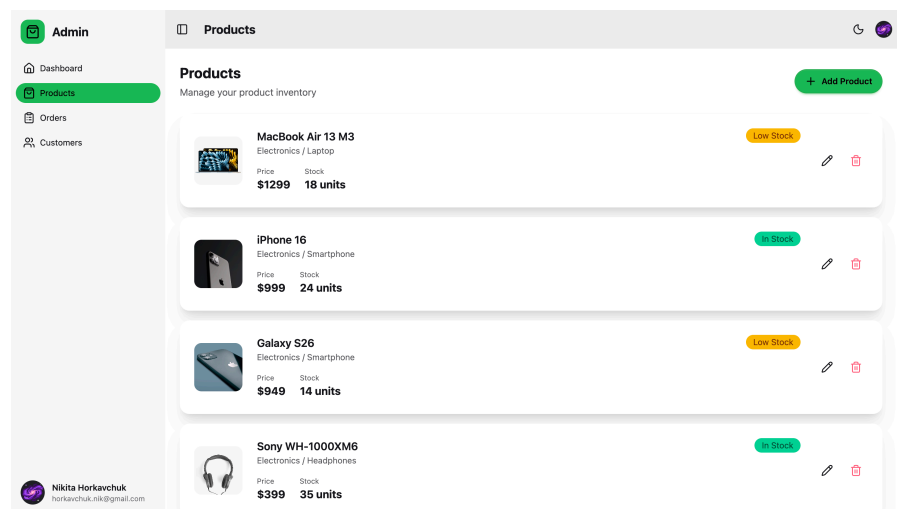


Рисунок 3.6 – Экран управления товарами в административной панели

Также в административной панели реализован раздел заказов. Он позволяет просматривать оформленные пользователями заказы, их состав, итоговую стоимость, адрес доставки и текущий статус. Администратор может использовать этот раздел для контроля обработки заказов и изменения их состояния. Обновлённый статус заказа отображается пользователю в мобильном приложении.

Раздел клиентов предназначен для просмотра информации о зарегистрированных пользователях. В нём администратор может видеть основные данные пользователей и связанную с ними активность. При этом чувствительные платёжные данные в панели не отображаются, так как обработка оплаты выполняется через внешний платёжный сервис.

Работа административной панели построена через взаимодействие с backend API. При открытии разделов веб-приложение отправляет запросы на сервер, получает данные и отображает их в интерфейсе. При создании товара, изменении заказа или получении статистики все операции также проходят через серверную

часть. Это позволяет централизованно проверять права доступа, обрабатывать данные и сохранять изменения в MongoDB.

В интерфейсе административной панели предусмотрены состояния загрузки, ошибок и успешного выполнения операций. Например, при загрузке списка товаров отображается состояние ожидания, а после успешного добавления товара список обновляется. Такие элементы делают работу администратора более понятной и снижают вероятность ошибок.

Реализованная административная панель обеспечивает операционную часть системы. Администратор может управлять каталогом товаров, контролировать заказы, просматривать клиентов и анализировать основные показатели работы платформы. Все изменения, выполненные через административную панель, сохраняются на сервере и отображаются в мобильном приложении.

3.6 Интеграция внешних сервисов

В рамках разработки системы электронной коммерции были использованы внешние сервисы, которые обеспечивают авторизацию пользователей, обработку платежей, хранение изображений, мониторинг ошибок, выполнение фоновых задач и работу модуля интеллектуального подбора товаров. Использование таких сервисов позволяет не реализовывать все механизмы с нуля, а подключить готовые решения для наиболее важных частей приложения. В проекте внешние сервисы используются как в мобильном приложении, так и в серверной части и административной панели. Согласно общей структуре проекта, система включает мобильное приложение, backend API, административную панель, авторизацию, платежи, хранение изображений и AI-подбор товаров.

Одним из ключевых внешних сервисов является Clerk, который используется для авторизации пользователей. С его помощью реализован вход в систему, включая авторизацию через Google и Apple. Это упрощает процесс регистрации и входа для пользователя, так как ему не требуется создавать отдельную учётную запись вручную. В мобильном приложении Clerk применяется для получения данных авторизованного пользователя, а на серверной стороне — для проверки доступа к защищённым маршрутам.

Интеграция Clerk важна не только для входа пользователя, но и для связи его действий с записью в базе данных. После авторизации backend может сопоставить пользователя с его профилем в MongoDB по уникальному идентификатору. Благодаря этому корзина, избранные товары, адреса доставки и заказы сохраняются для конкретного пользователя. Также Clerk используется в административной

панели, где доступ к управлению товарами, заказами и клиентами должен быть ограничен только пользователями с соответствующими правами.

Для реализации оплаты заказов используется Stripe. Данный сервис отвечает за обработку платежей и позволяет не хранить чувствительные данные банковских карт внутри собственной системы. Мобильное приложение инициирует процесс оплаты, однако основная логика создания платёжной сессии выполняется на серверной стороне. Это необходимо для безопасности, так как итоговая сумма заказа должна проверяться backend-сервером, а не передаваться напрямую с клиента.

После успешной оплаты Stripe отправляет событие на webhook серверной части. Webhook используется для подтверждения факта оплаты и дальнейшего изменения состояния заказа. Такой подход является более надёжным, чем простая обработка ответа от мобильного приложения, поскольку подтверждение приходит напрямую от платёжного сервиса. После получения подтверждённого события сервер может создать заказ, изменить его статус и очистить корзину пользователя.

Для хранения изображений товаров используется Cloudinary. Изображения не сохраняются напрямую в базе данных, так как это увеличило бы размер документов и усложнило бы работу с медиафайлами. Вместо этого административная панель передаёт изображение на сервер, backend загружает его в Cloudinary, а в базе данных сохраняется ссылка на полученный файл. Затем эта ссылка используется в мобильном приложении для отображения карточек товаров и подробной информации о продукции.

Такой подход упрощает управление изображениями и снижает нагрузку на серверную часть. Администратор может добавлять или изменять изображения товаров через панель управления, а пользователь мобильного приложения получает уже готовые ссылки на изображения. Это делает каталог товаров более удобным для сопровождения и масштабирования.

Для реализации интеллектуального подбора товаров используется Gemini API. Данный сервис применяется для обработки пользовательских запросов, сформулированных на естественном языке. Пользователь может написать, какой товар ему нужен, не указывая точные фильтры или технические характеристики. Backend получает этот запрос, выполняет предварительный поиск по каталогу товаров и затем использует Gemini для повышения качества рекомендации. В списке источников для проекта Gemini API указан как источник для описания интеграции модуля интеллектуального подбора товаров [7].

Важно, что мобильное приложение не обращается к Gemini напрямую. Запрос пользователя сначала отправляется на backend, где формируется контекст для AI-модели. Это позволяет скрыть ключи доступа к внешнему сервису, контролировать объём передаваемых данных и обрабатывать возможные ошибки.

Если AI-сервис временно недоступен, система может использовать fallback-логику и вернуть результат на основе локального поиска по каталогу.

Для обработки отдельных событий и фоновых задач используется Inngest. Этот сервис позволяет выносить часть процессов из основного пользовательского сценария. Например, фоновые задачи могут использоваться для обработки событий, связанных с пользователями, заказами или другими действиями в системе. Такой подход помогает не перегружать основные API-запросы и делает работу backend более устойчивой.

Также в проекте используется Sentry, предназначенный для мониторинга ошибок. Он помогает фиксировать исключения и проблемы, возникающие в мобильном приложении, административной панели или серверной части. Это особенно важно после развертывания системы, когда ошибки могут возникать уже в реальных пользовательских сценариях. Наличие мониторинга позволяет быстрее обнаруживать проблемы и исправлять их. В списке источников Sentry указан как сервис для настройки мониторинга ошибок и исключений в React Native-приложениях [14].

Все внешние сервисы подключаются через переменные окружения. В них хранятся ключи доступа, идентификаторы проектов, секреты webhook и другие параметры конфигурации. Такой подход позволяет не размещать конфиденциальные данные в исходном коде и упрощает перенос проекта между локальной средой разработки и средой развертывания. Для каждого окружения могут использоваться свои значения переменных, например отдельные ключи для разработки и production-среды.

Интеграция внешних сервисов позволила расширить функциональные возможности системы без чрезмерного усложнения собственной серверной логики. Clerk обеспечивает авторизацию, Stripe — оплату заказов, Cloudinary — хранение изображений, Gemini — интеллектуальный подбор товаров, Inngest — обработку фоновых событий, а Sentry — мониторинг ошибок. Совместное использование этих сервисов обеспечивает работу ключевых функций электронной торговой платформы и повышает надёжность разработанного приложения.

Развертывание системы является завершающим этапом разработки, на котором отдельные компоненты проекта подготавливаются к работе вне локальной среды. Разработанная система включает мобильное приложение, серверную API-часть и административную панель. Эти компоненты связаны между собой через backend API, который обеспечивает работу с базой данных, авторизацией, платежами, изображениями и модулем интеллектуального подбора товаров.

Для развертывания серверной части и административной панели была использована платформа Vercel. Данная платформа позволяет публиковать веб-приложения и backend-приложения, а также настраивать их работу через

переменные окружения и конфигурационные файлы. Согласно официальной документации Vercel, Express-приложение при развертывании на данной платформе преобразуется в Vercel Function, что позволяет использовать серверную часть в облачной среде [15].

Серверная часть системы была подготовлена к размещению на Vercel как отдельное backend-приложение. Для её работы были настроены переменные окружения, необходимые для подключения к MongoDB, Clerk, Stripe, Cloudinary, Gemini API, Inngest и Sentry. Хранение таких параметров вне исходного кода позволяет не размещать конфиденциальные данные в репозитории и упрощает перенос проекта между локальной средой разработки и production-средой.

Для корректной маршрутизации запросов к серверной части используется конфигурация Vercel. Она позволяет направлять API-запросы к backend-приложению и обеспечивать обработку маршрутов Express. Также через настройки окружения задаются адрес базы данных, ключи внешних сервисов, секреты webhook и другие параметры, необходимые для работы системы после публикации.

Административная панель также была развернута как отдельное веб-приложение. После публикации она обращается к серверной части через заранее указанный адрес API. Такой подход позволяет отделить интерфейс администратора от backend-логики и при необходимости обновлять административную панель и серверную часть независимо друг от друга. Через административную панель администратор может управлять товарами, заказами, клиентами и просматривать статистику работы платформы.

Мобильное приложение разрабатывалось на базе React Native и Expo. Для его работы после развертывания необходимо указать адрес серверной части, к которому приложение обращается при загрузке каталога товаров, работе с корзиной, оформлении заказов, авторизации пользователя и выполнении интеллектуального подбора товаров. Благодаря этому мобильное приложение может взаимодействовать с удалённым backend API, а не только с локальным сервером разработки.

Отдельное внимание при развертывании уделялось настройке внешних сервисов. Для Stripe необходимо указать webhook-адрес, на который платёжный сервис отправляет события об успешной оплате. Для Cloudinary задаются параметры доступа к облачному хранилищу изображений. Для Gemini API используется ключ доступа, необходимый серверной части для обработки запросов к модулю интеллектуального подбора товаров.

После развертывания выполняется проверка основных пользовательских сценариев. Проверяется авторизация, загрузка каталога товаров, добавление товаров в корзину, оформление заказа, работа оплаты, отображение заказов, загрузка изображений и доступ к административной панели. Также проверяется работа

интеллектуального подбора товаров, так как данный модуль зависит от корректного взаимодействия backend API, базы данных и Gemini API.

Развернутая система становится доступной для использования вне локальной среды разработки. Мобильное приложение, административная панель и серверная часть работают как единая система, а внешние сервисы обеспечивают авторизацию, оплату, хранение изображений, мониторинг ошибок, фоновые задачи и интеллектуальный подбор товаров.

4. ТЕСТИРОВАНИЕ СИСТЕМЫ

Тестирование разработанной системы проводилось с целью проверки корректности работы основных компонентов приложения и подтверждения того, что реализованные функции соответствуют поставленным требованиям. В процессе разработки были подготовлены и выполнены тесты, направленные на проверку мобильного приложения, серверной части, административной панели и модуля интеллектуального подбора товаров.

Наличие тестов в приложении позволило выявлять возможные ошибки на ранних этапах разработки и быстрее проверять корректность изменений после доработки отдельных функций. Это особенно важно для системы электронной коммерции, поскольку отдельные части приложения тесно связаны между собой: действия пользователя в мобильном приложении приводят к отправке запросов на сервер, изменению данных в базе данных, обновлению корзины, созданию заказов и отображению информации в административной панели.

Основной целью тестирования являлась проверка стабильности работы системы в типовых пользовательских сценариях. К таким сценариям относятся авторизация пользователя, просмотр каталога товаров, добавление товаров в корзину и избранное, оформление заказа, а также взаимодействие администратора с товарами и заказами. Дополнительно проверялась работа модуля интеллектуального подбора товаров, который должен корректно обрабатывать текстовый запрос пользователя и возвращать подходящие товары. В рамках тестирования были поставлены следующие задачи:

- проверить корректность работы основных экранов мобильного приложения;
- проверить выполнение пользовательских сценариев покупки товара;
- проверить корректность отправки запросов с клиента на сервер;
- проверить обработку успешных и ошибочных ответов API;
- проверить сохранение и обновление данных в базе данных;
- проверить работу защищённых маршрутов и ограничение доступа;
- проверить функции административной панели;
- проверить корректность работы AI-модуля подбора товаров;
- убедиться, что после внесения изменений ранее реализованные функции продолжают работать корректно.

Тестирование включало как ручную проверку пользовательских сценариев, так и выполнение тестов, добавленных в приложение. Ручное тестирование использовалось для проверки интерфейса, навигации и общего поведения системы с точки зрения пользователя. Тесты в приложении применялись для проверки отдельных функций, обработки данных, корректности запросов и взаимодействия компонентов.

Проведённое тестирование позволило подтвердить работоспособность основных функций приложения, проверить взаимодействие между клиентской и серверной частями, а также убедиться в корректной работе системы в рамках MVP.

Тестирование мобильного приложения проводилось с целью проверки корректности работы клиентской логики, обработки данных и взаимодействия пользовательской части приложения с серверным API. Так как мобильное приложение является основным интерфейсом для покупателя, особое внимание было уделено тем функциям, которые используются в процессе выбора товара, добавления его в корзину, работы со списком избранного и оформления заказа.

В рамках тестирования проверялись не только визуальные элементы приложения, но и внутренняя логика, от которой зависит корректность отображения данных. Для этого в мобильном приложении была настроена инфраструктура автоматического тестирования с использованием Jest и ts-jest. Такой подход позволил проверять отдельные функции и хуки без необходимости каждый раз вручную проходить весь пользовательский сценарий. Для запуска тестов были добавлены команды `npm run test` и `npm run test:watch`, что упростило проверку приложения во время разработки и после внесения изменений в код.

Результат выполнения автоматических тестов представлен на рисунке 4.1. На данном этапе было успешно выполнено 3 набора тестов, включающих 8 отдельных тестовых случаев. Все тесты были пройдены без ошибок, что подтверждает корректную работу проверяемых частей мобильного приложения.

```
> mobile@1.0.0 test
> jest --runInBand

PASS  __tests__/useCart.test.ts
PASS  __tests__/useAddresses.test.ts
PASS  __tests__/useWishlist.test.ts
PASS  __tests__/queryHooks.test.ts
PASS  __tests__/useReviews.test.ts
PASS  __tests__/utils.test.ts

Test Suites: 6 passed, 6 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        3.341 s
Ran all test suites.
n.gorkavchuk@Mac expo-ecommerce %
```

Рисунок 4.1 — Результат выполнения тестов мобильного приложения

В первую очередь были протестированы вспомогательные функции, расположенные в файле `lib/utils.ts`. Данные функции используются в разных частях интерфейса и отвечают за подготовку данных к отображению пользователю. Проверка функции `capitalizeFirstLetter` была необходима для подтверждения корректной капитализации строк, которые могут использоваться при выводе названий, статусов или других текстовых значений. Также была протестирована функция `formatDate`, отвечающая за преобразование даты в ожидаемый формат `en-`

US. Корректное отображение даты важно для экранов, связанных с заказами, историей покупок и статусами доставки.

Дополнительно была проверена функция `getStatusColor`, которая используется для визуального отображения состояния заказа. В ходе тестирования проверялись основные статусы, используемые в приложении, включая `delivered`, `shipped` и `pending`. Также был проверен вариант с неизвестным статусом, для которого функция должна возвращать `fallback`-значение. Это позволяет избежать некорректного отображения интерфейса в ситуации, когда приложение получает неожиданный или новый статус заказа с серверной части.

Отдельное внимание было уделено тестированию логики корзины, реализованной в хуке `hooks/useCart.ts`. Корзина является одной из ключевых частей мобильного приложения электронной коммерции, поэтому ошибки в её работе могут напрямую повлиять на процесс оформления заказа. В ходе тестирования проверялась обработка элементов корзины, подсчёт итоговой стоимости товаров, подсчёт общего количества товаров, а также корректность выполнения API-запросов к серверной части.

Одним из важных тестовых сценариев стала проверка фильтрации некорректных элементов корзины. В приложении может возникнуть ситуация, когда в массиве `cart.items` находится элемент без связанного объекта `product`. Например, это может произойти при удалении товара из базы данных или при получении неполных данных от сервера. В таком случае мобильное приложение не должно аварийно завершать работу или отображать некорректную карточку товара. Проведённое тестирование подтвердило, что такие элементы корректно исключаются из отображения и не участвуют в расчёте итоговой стоимости.

Также была проверена корректность вычисляемых значений `cartTotal` и `cartItemCount`. Значение `cartTotal` используется для отображения общей суммы заказа, а `cartItemCount` — для отображения количества товаров в корзине. Эти показатели должны обновляться при добавлении товара, изменении количества или удалении позиции из корзины. Проверка подтвердила, что расчёт выполняется корректно и соответствует текущему состоянию корзины.

Пример тестирования логики корзины представлен на рисунке 4.2. Данный фрагмент демонстрирует проверку клиентской логики, связанной с обработкой данных корзины и взаимодействием с серверным API.

```

PASS  __tests__/useCart.test.ts
PASS  __tests__/utils.test.ts
PASS  __tests__/useWishlist.test.ts
PASS  __tests__/useReviews.test.ts
PASS  __tests__/useAddresses.test.ts
PASS  __tests__/queryHooks.test.ts

Test Suites: 6 passed, 6 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        3.346 s
Ran all test suites related to changed files.

Watch Usage: Press w to show more.

```

Рисунок 4.2 — Тестирование логики корзины мобильного приложения

В ходе тестирования корзины также проверялась корректность API-вызовов, которые выполняются при действиях пользователя. Были проверены запросы на получение корзины, добавление товара, изменение количества товара, удаление отдельной позиции и очистку всей корзины. Для этого тестировались вызовы GET /cart, POST /cart, PUT /cart/:id, DELETE /cart/:id и DELETE /cart. Такая проверка позволила убедиться, что клиентская часть обращается к нужным маршрутам серверного API и использует корректные методы запросов.

Дополнительно проверялась инвалидация кэша ["cart"] после успешного выполнения мутаций. Это важно, поскольку мобильное приложение использует кэширование данных, и после изменения корзины пользователь должен видеть актуальное состояние, а не устаревшие данные. Проверка показала, что после добавления, изменения или удаления товара кэш корзины корректно инвалидируется, после чего приложение может получить обновлённые данные с сервера.

Кроме корзины, было проведено тестирование логики избранных товаров, реализованной в хуке hooks/useWishlist.ts. Список избранного используется для сохранения товаров, к которым пользователь может вернуться позже. В рамках тестирования проверялась функция isInWishlist(productId), которая определяет, находится ли конкретный товар в списке избранного. Корректная работа этой функции важна для отображения состояния кнопки избранного на карточке товара и в списке товаров.

Также была протестирована функция toggleWishlist, отвечающая за переключение состояния товара в избранном. Если товар уже находится в списке избранного, функция должна инициировать его удаление. Если товар отсутствует в списке, функция должна выполнить добавление. Такой подход позволяет использовать одну и ту же кнопку в интерфейсе для двух действий, что упрощает пользовательское взаимодействие. Тестирование подтвердило, что логика переключения работает корректно и соответствует ожидаемому поведению.

После успешного добавления или удаления товара из избранного также проверялась инвалидация кэша ["wishlist"]. Благодаря этому после изменения списка избранных товаров интерфейс получает актуальные данные и корректно отображает состояние товара. Пример тестирования логики избранных товаров представлен на рисунке 4.3.

```

n.gorkavchuk@Nikitas-MacBook-Air mobile % npm run test -- __tests__/useWishlist.test.ts
> mobile@1.0.0 test
> jest --runInBand __tests__/useWishlist.test.ts

PASS __tests__/useWishlist.test.ts
  useWishlist
    ✓ checks if product exists in wishlist and exposes count (1 ms)
    ✓ toggles wishlist by calling add/remove mutate (1 ms)
    ✓ invalidates wishlist after add/remove success

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:  0 total
Time:        1.061 s
Ran all test suites matching __tests__/useWishlist.test.ts.
n.gorkavchuk@Nikitas-MacBook-Air mobile %

```

Рисунок 4.3 — Тестирование логики избранных товаров

Проведённое тестирование мобильного приложения подтвердило корректную работу основных частей клиентской логики. Были проверены вспомогательные функции, логика корзины, взаимодействие с API, работа с кэшем и логика избранных товаров. Успешное прохождение всех добавленных тестов показывает, что проверяемые функции работают стабильно и могут использоваться в рамках основного пользовательского сценария приложения.

Наличие автоматических тестов также упрощает дальнейшую разработку мобильного приложения. При изменении логики корзины, избранного или вспомогательных функций тесты позволяют быстро проверить, не была ли нарушена уже реализованная функциональность. Это особенно важно для проекта электронной коммерции, где ошибки в расчёте стоимости, количестве товаров или состоянии корзины могут привести к некорректному оформлению заказа и ухудшению пользовательского опыта.

Тестирование серверной части проводилось с целью проверки корректности работы API, обработки запросов от мобильного приложения и административной панели, а также взаимодействия сервера с базой данных и внешними сервисами. Серверная часть является центральным компонентом системы, поскольку именно она отвечает за получение и сохранение данных, выполнение бизнес-логики, проверку прав доступа, обработку заказов, работу с корзиной, интеграцию с платёжной системой и передачу данных клиентским приложениям.

В ходе тестирования проверялась работа основных маршрутов серверного API. Особое внимание было уделено тем запросам, которые используются в ключевых пользовательских сценариях: получение списка товаров, просмотр информации о конкретном товаре, работа с корзиной, оформление заказа, получение пользовательских данных, а также выполнение административных операций.

Первым этапом было протестировано получение данных о товарах. Проверялось, что сервер корректно возвращает список товаров, включая название, описание, цену, количество на складе, категорию, подкатегорию, характеристики, изображения и рейтинг. Также проверялось получение информации о конкретном товаре по идентификатору. В случае существующего товара сервер должен возвращать полные данные, необходимые для отображения карточки товара в мобильном приложении. При передаче некорректного или несуществующего идентификатора сервер должен возвращать ошибку, не нарушая работу приложения. Пример проверки запроса на получение списка товаров представлен на рисунке 4.4.

```

n.gorkavchuk@Nikitas-MacBook-Air expo-ecommerce % npm run test:server --prefix backend -- -t "returns products list via GET /api/products"

> backend@1.0.0 test:server
> vitest run tests/server-api.test.js -t returns products list via GET /api/products

RUN v3.2.4 /Users/n.gorkavchuk/Desktop/expo-ecommerce/backend
✓ tests/server-api.test.js (10 tests | 9 skipped) 3ms
  ✓ Server API routes > Products API > returns products list via GET /api/products 3ms
  ↓ Server API routes > Products API > blocks request to protected route without token
  ↓ Server API routes > Products API > returns 404 when token is valid but user is missing in DB
  ↓ Server API routes > Cart API > returns cart via GET /api/cart
  ↓ Server API routes > Cart API > adds item via POST /api/cart
  ↓ Server API routes > Cart API > updates item quantity via PUT /api/cart/:productId
  ↓ Server API routes > Cart API > removes item via DELETE /api/cart/:productId
  ↓ Server API routes > Cart API > clears cart via DELETE /api/cart
  ↓ Server API routes > Orders API > creates order and decrements stock via POST /api/orders
  ↓ Server API routes > Orders API > returns user orders with review status via GET /api/orders

Test Files  1 passed (1)
Tests       1 passed | 9 skipped (10)
Start at    21:20:02
Duration    489ms (transform 71ms, setup 0ms, collect 298ms, tests 3ms, environment 0ms, prepare 41ms)

n.gorkavchuk@Nikitas-MacBook-Air expo-ecommerce %

```

Рисунок 4.4 — Проверка получения списка товаров через серверный API

Следующим этапом было проведено тестирование маршрутов, связанных с корзиной пользователя. В рамках проверки анализировалась работа запросов на получение корзины, добавление товара, изменение количества товара, удаление отдельной позиции и полную очистку корзины. Сервер должен корректно определять пользователя, связывать корзину с его учётной записью и возвращать актуальное состояние корзины после каждого изменения. Также проверялось, что при добавлении товара сервер учитывает переданный идентификатор товара и количество, а при изменении корзины возвращает обновлённые данные.

Отдельное внимание было уделено обработке некорректных данных. Например, проверялись ситуации, когда в запросе отсутствует идентификатор товара, передано некорректное количество или пользователь пытается изменить товар, которого нет в корзине. В таких случаях сервер должен возвращать понятный ответ с ошибкой и соответствующим HTTP-статусом. Это важно, поскольку мобильное приложение должно иметь возможность корректно обработать ошибку и не отображать пользователю неверные данные.

Также было протестировано оформление заказа. Проверялось, что сервер получает данные о товарах, адресе доставки и пользователе, после чего создаёт заказ с корректной структурой. При оформлении заказа серверная часть должна

использовать данные из базы, а не полагаться только на информацию, переданную с клиента. Такой подход снижает риск изменения цены или количества товаров на стороне клиента. После успешного оформления заказа данные должны сохраняться в базе данных и становиться доступными для просмотра как пользователем в мобильном приложении, так и администратором в панели управления.

Пример проверки запроса, связанного с работой корзины или созданием заказа, представлен на рисунке 4.5.

```
n.gorkavchuk@Nikitas-MacBook-Air expo-ecommerce % npm run test:server --prefix backend -- -t "Cart API|Orders API"
> backend@1.0.0 test:server
> vitest run tests/server-api.test.js -t Cart API|Orders API

 RUN v3.2.4 /Users/n.gorkavchuk/Desktop/expo-ecommerce/backend
 ✓ tests/server-api.test.js (10 tests | 3 skipped) 6ms
   ✓ Server API routes > Products API > returns products list via GET /api/products
   ✓ Server API routes > Products API > blocks request to protected route without token
   ✓ Server API routes > Products API > returns 404 when token is valid but user is missing in DB
   ✓ Server API routes > Cart API > returns cart via GET /api/cart 3ms
   ✓ Server API routes > Cart API > adds item via POST /api/cart 1ms
   ✓ Server API routes > Cart API > updates item quantity via PUT /api/cart/:productId 0ms
   ✓ Server API routes > Cart API > removes item via DELETE /api/cart/:productId 0ms
   ✓ Server API routes > Cart API > clears cart via DELETE /api/cart 0ms
   ✓ Server API routes > Orders API > creates order and decrements stock via POST /api/orders 1ms
   ✓ Server API routes > Orders API > returns user orders with review status via GET /api/orders 1ms

Test Files  1 passed (1)
Tests       7 passed | 3 skipped (10)
Start at    21:20:59
Duration    484ms (transform 69ms, setup 0ms, collect 295ms, tests 6ms, environment 0ms, prepare 35ms)

n.gorkavchuk@Nikitas-MacBook-Air expo-ecommerce %
```

Рисунок 4.5 — Проверка серверной обработки корзины и заказов

Важной частью тестирования серверной части стала проверка защищённых маршрутов. Поскольку в системе используется авторизация, часть запросов должна быть доступна только авторизованным пользователям. Дополнительно административные маршруты должны быть доступны только пользователям с соответствующей ролью. В ходе тестирования проверялось, что при отсутствии токена авторизации сервер ограничивает доступ к защищённым данным, а при наличии корректного токена разрешает выполнение запроса. Для административных маршрутов дополнительно проверялось, что обычный пользователь не может получить доступ к функциям управления товарами, заказами и клиентами.

Также проводилась проверка маршрутов административной панели. Сервер должен корректно обрабатывать запросы на добавление, редактирование и удаление товаров, получение списка заказов, изменение статуса заказа и получение статистики. При этом данные, изменённые через административную панель, должны отображаться в мобильном приложении. Например, если администратор изменяет количество товара на складе или обновляет его описание, пользователь должен получать уже актуальную информацию при следующей загрузке каталога.

Дополнительно была проверена обработка ошибок на стороне сервера. В процессе тестирования анализировались ответы при некорректных запросах, отсутствии обязательных полей, неверных идентификаторах и попытке доступа к закрытым маршрутам. Сервер должен возвращать корректные HTTP-статусы и

сообщения об ошибках. Это позволяет клиентской части правильно реагировать на проблемы и отображать пользователю соответствующие уведомления.

Отдельно проверялась работа серверной части при взаимодействии с внешними сервисами. Поскольку система использует интеграции для авторизации, оплаты, хранения изображений и интеллектуального подбора товаров, сервер должен корректно обрабатывать как успешные ответы, так и возможные ошибки внешних сервисов. Например, при работе с оплатой важно, чтобы заказ создавался только после подтверждения успешного платежа. При загрузке изображений товаров сервер должен сохранять корректные ссылки на изображения, которые затем используются в мобильном приложении и административной панели.

Пример проверки защищённого или административного маршрута представлен на рисунке 4.6.

```
● n.gorkavchuk@Nikitas-MacBook-Air expo-e-commerce % npm run test:server --prefix backend -- -t "blocks request to protected route without token|returns 404 when token is valid but user is missing in DB"

> backend@1.0.0 test:server
> vitest run tests/server-api.test.js -t blocks request to protected route without token|returns 404 when token is valid but user is missing in DB

RUN v3.2.4 /Users/n.gorkavchuk/Desktop/expo-e-commerce/backend
✓ tests/server-api.test.js (10 tests | 8 skipped) 3ms
  ✓ Server API routes > Products API > returns products list via GET /api/products
  ✓ Server API routes > Products API > blocks request to protected route without token 2ms
  ✓ Server API routes > Products API > returns 404 when token is valid but user is missing in DB 0ms
  ✓ Server API routes > Cart API > returns cart via GET /api/cart
  ✓ Server API routes > Cart API > adds item via POST /api/cart
  ✓ Server API routes > Cart API > updates item quantity via PUT /api/cart/:productId
  ✓ Server API routes > Cart API > removes item via DELETE /api/cart/:productId
  ✓ Server API routes > Cart API > clears cart via DELETE /api/cart
  ✓ Server API routes > Orders API > creates order and decrements stock via POST /api/orders
  ✓ Server API routes > Orders API > returns user orders with review status via GET /api/orders

Test Files 1 passed (1)
Tests 2 passed | 8 skipped (10)
Start at 21:21:37
Duration 491ms (transform 70ms, setup 0ms, collect 297ms, tests 3ms, environment 0ms, prepare 40ms)

● n.gorkavchuk@Nikitas-MacBook-Air expo-e-commerce %
```

Рисунок 4.6 — Проверка защищённого маршрута серверного API

Результаты тестирования показали, что серверная часть корректно обрабатывает основные запросы, необходимые для работы мобильного приложения и административной панели. Были проверены маршруты для работы с товарами, корзиной, заказами, пользователями и административными функциями. Сервер корректно взаимодействует с базой данных, возвращает данные в ожидаемом формате, обрабатывает ошибочные ситуации и ограничивает доступ к защищённым маршрутам.

Проведённое тестирование подтвердило, что серверная часть выполняет роль связующего компонента системы и обеспечивает стабильное взаимодействие между мобильным приложением, административной панелью, базой данных и внешними сервисами. Это позволяет использовать реализованный API в рамках MVP и продолжать дальнейшее развитие системы без нарушения основной бизнес-логики.

Результаты тестирования показали, что серверная часть корректно обрабатывает основные запросы, необходимые для работы мобильного приложения

и административной панели. Были проверены маршруты для работы с товарами, корзиной, заказами, пользователями и административными функциями. Сервер корректно взаимодействует с базой данных, возвращает данные в ожидаемом формате, обрабатывает ошибочные ситуации и ограничивает доступ к защищённым маршрутам.

Тестирование административной панели проводилось с целью проверки корректности работы интерфейса администратора, а также основных функций управления системой. Данная часть приложения предназначена для контроля товаров, заказов, клиентов и просмотра общей статистики, поэтому её стабильная работа является важным условием корректного функционирования всей системы электронной коммерции. Если мобильное приложение используется покупателем, то административная панель используется для управления внутренними процессами и актуализации данных, которые затем отображаются пользователям.

В ходе тестирования проверялась общая работоспособность административной панели, корректность переходов между разделами, загрузка данных с серверной части и выполнение основных действий администратора. Особое внимание уделялось тому, чтобы интерфейс корректно отображал данные, полученные с сервера, а также обновлялся после выполнения операций. Проверялись разделы, связанные со статистикой, товарами, заказами и клиентами. Это позволило убедиться, что администратор может получить доступ к основной информации о состоянии системы и выполнять необходимые действия без возникновения ошибок.

Также проверялась работа защищённых разделов административной панели. Поскольку панель содержит функции, недоступные обычному пользователю, доступ к ней должен предоставляться только авторизованному администратору. В процессе тестирования оценивалось, что пользователь без соответствующих прав не может перейти к административным функциям или выполнить действия, связанные с управлением товарами и заказами. Это необходимо для защиты данных и предотвращения несанкционированного изменения информации в системе.

Отдельно была проверена корректность отображения статистической информации. Административная панель должна получать с сервера данные о количестве товаров, заказов, клиентов и других ключевых показателях. В ходе проверки анализировалось, что данные загружаются без ошибок, отображаются в интерфейсе в понятном виде и соответствуют информации, которая хранится на серверной стороне. Данный раздел важен для администратора, так как позволяет быстро оценить текущее состояние магазина.

В процессе тестирования также проверялась работа раздела управления товарами. Администратор должен иметь возможность просматривать список товаров, добавлять новые позиции, редактировать существующие данные и удалять

товары при необходимости. При проверке учитывалось, что после изменения информации в административной панели данные должны сохраняться на сервере и отображаться в мобильном приложении. Например, при изменении названия, цены, описания, изображения или количества товара обновлённые данные должны быть доступны пользователю при следующей загрузке каталога.

Особое внимание уделялось формам добавления и редактирования товаров. Проверялась корректность заполнения полей, обработка обязательных данных и сохранение изменений. Если данные введены корректно, панель должна отправлять запрос на сервер и отображать обновлённую информацию. Если данные заполнены неправильно или отсутствуют обязательные поля, интерфейс должен корректно обработать такую ситуацию и не допустить сохранения некорректной записи. Такая проверка важна, поскольку ошибки при создании или редактировании товаров могут привести к неправильному отображению каталога в мобильном приложении.

Также была проведена проверка раздела заказов. В административной панели должна отображаться информация о заказах пользователей, их составе, стоимости и текущем статусе. Проверялось, что данные заказов корректно загружаются с сервера и отображаются в интерфейсе администратора. Дополнительно оценивалась возможность изменения статуса заказа. После изменения статуса данные должны сохраняться и становиться доступными пользователю в мобильном приложении. Это позволяет поддерживать актуальность информации о доставке и состоянии заказа.

Кроме того, тестировалось отображение информации о клиентах. Административная панель должна позволять просматривать данные пользователей, которые взаимодействуют с системой. Проверка данного раздела была необходима для подтверждения того, что данные клиентов корректно передаются с серверной части и отображаются без ошибок. Это помогает администратору контролировать активность пользователей и анализировать работу системы.

В рамках тестирования также оценивалась обработка возможных ошибок. Проверялось поведение панели при отсутствии данных, ошибке загрузки, некорректном ответе сервера или попытке выполнить действие без необходимых прав. Интерфейс не должен аварийно завершать работу или отображать некорректные данные. Вместо этого пользователь должен видеть понятное состояние интерфейса, позволяющее продолжить работу после устранения проблемы.

Для проверки административной панели были выполнены общие тесты, направленные на оценку доступности основных разделов, корректности загрузки данных и выполнения базовых операций управления. Результат выполнения тестов административной панели представлен на рисунке 4.7. На данном рисунке можно

показать общий результат проверки, без детального отображения внутренней структуры тестов и названий отдельных сценариев.

```
● n.gorkavchuk@Nikitas-MacBook-Air expo-ecommerce % npm run test --prefix admin

> admin@0.0.0 test
> vitest run

 RUN  v4.1.5 /Users/n.gorkavchuk/Desktop/expo-ecommerce/admin
 ✓ src/__tests__/theme.test.js (4 tests) 5ms
 ✓ src/__tests__/authToken.test.js (4 tests) 5ms
 ✓ src/__tests__/utils.test.js (4 tests) 21ms
 ✓ src/__tests__/api.test.js (8 tests) 9ms
 ✓ src/__tests__/useAdminTheme.test.jsx (2 tests) 11ms
 ✓ src/__tests__/App.test.jsx (5 tests) 30ms

Test Files  6 passed (6)
Tests       27 passed (27)
Start at    21:37:37
Duration    983ms (transform 212ms, setup 371ms, import 384ms, tests 81ms, environment 3.94s)

○ n.gorkavchuk@Nikitas-MacBook-Air expo-ecommerce %
```

Рисунок 4.7 — Общий результат тестирования административной панели

Результаты тестирования показали, что административная панель корректно выполняет основные задачи, предусмотренные в рамках разработанной системы. Интерфейс позволяет администратору просматривать статистику, управлять товарами, контролировать заказы и получать информацию о клиентах. Данные корректно загружаются с серверной части, изменения сохраняются в базе данных и становятся доступными другим компонентам системы. Проведённая проверка подтверждает, что административная панель готова к использованию в рамках MVP и обеспечивает базовый набор функций для управления электронной коммерцией.

Тестирование модуля интеллектуального подбора товаров проводилось с целью проверки корректности обработки пользовательских запросов, введённых в свободной форме. Данный модуль является одной из ключевых особенностей разработанной системы, так как позволяет пользователю искать товары не только через стандартные фильтры и категории, но и с помощью обычного текстового описания своих потребностей.

В ходе тестирования проверялось, насколько корректно система определяет смысл запроса пользователя и связывает его с товарами, представленными в каталоге. Пользователь может не знать точное название товара, его технические характеристики или категорию, поэтому модуль должен уметь обрабатывать более естественные формулировки. Например, вместо выбора конкретной категории пользователь может написать, что ему нужна игровая мышь, наушники для спорта или недорогой товар для повседневного использования.

Проверка выполнялась через интерфейс мобильного приложения. Пользователь вводил текстовый запрос в поле интеллектуального помощника, после чего приложение отправляло данные на серверную часть. Сервер обрабатывал запрос, выполнял поиск подходящих товаров в базе данных и возвращал результат

обратно в мобильное приложение. После этого пользователь получал ответ системы и список рекомендованных товаров.

Во время тестирования использовались разные типы пользовательских запросов. Проверялись простые запросы, в которых пользователь указывал только общий тип товара, например «хочу игровую мышку» или «нужны наушники». Также проверялись более подробные запросы, в которых были указаны дополнительные параметры: примерная цена, назначение товара, категория или особенности использования. Это позволило оценить, насколько корректно модуль работает как с короткими, так и с более развёрнутыми формулировками.

Отдельно проверялись неполные и обобщённые запросы. В таких случаях пользователь не указывает конкретный товар, а описывает только общую потребность. Например, запрос «хочу что-нибудь для спорта» не содержит точной категории или характеристик. При обработке таких запросов система должна подобрать наиболее подходящие товары из доступного каталога либо вернуть наиболее близкие варианты. Такая проверка важна, поскольку в реальном использовании пользователь не всегда формулирует запрос точно и технически корректно.

Также была проверена ситуация, когда по запросу пользователя в каталоге отсутствуют подходящие товары. В этом случае модуль не должен возвращать случайные или нерелевантные рекомендации. Система должна корректно обработать отсутствие результата и сообщить пользователю, что подходящие товары не найдены. Это позволяет избежать неправильных рекомендаций и сохранить понятное поведение приложения.

Дополнительно проверялась устойчивость работы модуля при ошибке внешнего AI-сервиса. Поскольку интеллектуальный подбор связан с использованием внешней модели, важно, чтобы сбой AI-запроса не нарушал работу всего приложения. При возникновении ошибки система должна использовать резервную логику поиска или корректно сообщать пользователю о невозможности выполнить интеллектуальный подбор. Такая проверка позволяет убедиться, что основной функционал каталога остаётся доступным даже при недоступности внешнего сервиса.

Результаты тестирования показали, что модуль интеллектуального подбора товаров корректно обрабатывает основные типы пользовательских запросов. Система способна работать с простыми, уточнёнными и обобщёнными формулировками, подбирать релевантные товары из каталога и корректно реагировать на отсутствие подходящих результатов. Проверка также подтвердила, что модуль не нарушает работу мобильного приложения при возникновении ошибок и может использоваться как дополнительный инструмент для более удобного поиска товаров.

По результатам проведённого тестирования была выполнена оценка работоспособности основных компонентов разработанной системы. Проверка охватывала мобильное приложение, серверную часть, административную панель и модуль интеллектуального подбора товаров. Основное внимание уделялось корректности выполнения пользовательских сценариев, стабильности работы приложения, обработке данных и взаимодействию между отдельными частями системы.

В ходе тестирования мобильного приложения была проверена клиентская логика, связанная с отображением данных, работой корзины и избранных товаров. Автоматические тесты подтвердили корректность работы вспомогательных функций, подсчёта итоговой стоимости корзины, количества товаров, API-вызовов и обновления кэша после изменения данных. Это показало, что основные элементы пользовательской логики работают стабильно и корректно обрабатывают данные, полученные от серверной части.

Тестирование серверной части показало, что API корректно обрабатывает основные запросы мобильного приложения и административной панели. Были проверены операции получения товаров, работы с корзиной, оформления заказов, обработки ошибок и ограничения доступа к защищённым маршрутам. Серверная часть корректно взаимодействует с базой данных, возвращает данные в ожидаемом формате и обеспечивает выполнение основной бизнес-логики приложения.

Проверка административной панели подтвердила, что администратор может выполнять основные действия, необходимые для управления системой. Панель корректно отображает статистическую информацию, данные о товарах, заказах и клиентах. Также было установлено, что изменения, выполняемые через административную панель, передаются на серверную часть и сохраняются в базе данных. Это позволяет поддерживать актуальность информации, отображаемой в мобильном приложении.

Отдельно были проанализированы результаты тестирования модуля интеллектуального подбора товаров. Проверка показала, что модуль корректно обрабатывает пользовательские запросы в свободной форме и способен подбирать товары на основе введённых предпочтений. Были проверены простые, уточнённые и обобщённые запросы. Также была оценена обработка ситуаций, когда подходящие товары отсутствуют или внешний AI-сервис недоступен. Это позволило убедиться, что модуль не нарушает работу приложения и может использоваться как дополнительный инструмент для поиска товаров.

В результате тестирования критических ошибок, препятствующих работе системы, выявлено не было. Основные функции мобильного приложения, серверной части, административной панели и AI-модуля работают корректно. Система

выполняет основные пользовательские сценарии, обеспечивает обмен данными между компонентами и сохраняет информацию в базе данных.

При этом тестирование также позволило определить направления для дальнейшего развития. В будущем целесообразно расширить количество автоматических тестов, добавить больше проверок пользовательского интерфейса, провести нагрузочное тестирование серверной части и дополнительно оценить качество рекомендаций интеллектуального модуля на большем количестве товаров и пользовательских запросов.

Результаты проведённого тестирования подтверждают, что разработанная система соответствует требованиям MVP. Реализованные компоненты работают согласованно, основные сценарии использования выполняются корректно, а приложение может быть использовано для демонстрации функциональности мобильной системы электронной коммерции с интеллектуальным подбором товаров.

					ДП.АС64.220038 – 06 81 00	Лист
Изм	Лист	№ докум	Подп.	Дата		67

5. РАСЧЕТ ТЕХНИКО-ЭКОНОМИЧЕСКИХ ПОКАЗАТЕЛЕЙ

5.1 Реализация функциональных возможностей мобильного приложения

Программное обеспечение, разрабатываемое в рамках данного дипломного проекта, предназначено для автоматизации процессов электронной коммерции посредством мобильного приложения, серверной части и административной панели. Система включает клиентскую часть для покупателей, серверную API-часть, базу данных, модуль обработки заказов, оплату товаров, управление товарами и модуль интеллектуального подбора товаров.

Создание данного программного продукта предполагает выполнение основных стадий проектирования, что соответствует второй группе сложности разработки. Основные этапы работы включают: составление технического задания, проектирование архитектуры системы, разработку мобильного приложения, серверной части, административной панели, базы данных и интеграцию внешних сервисов.

Последовательность расчётов:

1. расчет объёма функций программного модуля;
2. расчет полной себестоимости программного продукта;
3. расчет цены и прибыли по программному продукту;

5.2 Расчет объема функций

Средой разработки мобильной части программного продукта является Visual Studio Code с использованием React Native, Expo и TypeScript. Разработка серверной части осуществляется в среде Visual Studio Code на платформе Node.js с применением Express. В качестве системы управления базами данных используется MongoDB.

Общий объём ПО определяется по формуле (5.1) исходя из объёма функций, реализуемых программой:

$$V_0 = \sum_{i=1}^n V_i, \quad (5.1)$$

где V_0 – общий объем ПО;
 V_i – объем функций ПО;
 n – общее число функций.

Расчет общего объема программного продукта предполагает определение объема по каждой функции. В том случае, когда невозможно рассчитать точный объем функций, данный объем может быть получен на основании прогнозируемой оценки имеющихся фактических данных по аналогичным проектам или применением нормативов по каталогу функций.

Уточненный объем ПО определяется по формуле (5.2):

$$V_y = \sum_{i=1}^n V_{yi}, \quad (5.2)$$

где V_{yi} – уточненный объем отдельной функции в строках исходного кода.
 Перечень и объем функций ПО приведен в таблице 5.1.

Код функции	Наименование (содержание) функции	Объем функции (LOC)	
		По каталогу V_i	Уточненный V_{yi}
101	Авторизация и управление учетными записями пользователей	559	432
101	Организация ввода информации	130	850
102	Контроль, предварительная обработка и ввод информации	490	720
107	Организация ввода/вывода информации в интерактивном режиме	280	1050
201	Генерация структуры базы данных	3500	650
202	Формирование баз данных	1980	400
203	Обработка наборов и записей баз данных	2370	1350
207	Организация поиска и поиск в базе данных	4720	760
301	Формирование файлов	590	430
303	Обработка файлов	1050	520
401	Генерация рабочих программ	3120	420
507	Обеспечение интерфейса между компонентами	1680	840
701	Математическая статистика и прогнозирование	3780	900
803	Разработка системы оплаты услуг	820	620

Продолжение таблицы 5.1

Код функции	Наименование (содержание) функции	Объем функции (LOC)	
		По каталогу Vi	Уточнённый Vy _i
809	Создание системы управления контентом	970	800
602	Вспомогательные и сервисные программы	470	470
Итого		25950	10780

С учётом информации, указанной в таблице 5.1, уточнённый объём ПО составил 10780 строк кода вместо предполагаемого количества 25950 строк кода.

5.3 Расчет полной себестоимости программного модуля

С учётом информации, указанной в таблице 5.1, уточнённый объём ПО составил 10780 строк кода вместо предполагаемого количества 25950 строк кода.

Стоимостная оценка программного средства у разработчика предполагает составление сметы затрат, которая включает следующие статьи расходов:

1. отчисления на социальные нужды;
2. материалы и комплектующие изделия;
3. спецоборудование;
4. машинное время;
5. расходы на научные командировки;
6. прочие прямые расходы;
7. накладные расходы;
8. затраты на освоение и сопровождение программного средства.

Полная себестоимость разработки программного продукта рассчитывается как сумма расходов по всем статьям.

Основной статьёй расходов на создание программного продукта является заработная плата проекта: основная и дополнительная заработная плата разработчиков. В данном дипломном проекте три исполнителя: два инженера-программиста и научный руководитель.

Продолжительность разработки программного продукта — 90 дней.

Месячная тарифная ставка 1 разряда — 297 бел. руб.

Коэффициент премии — 1,5.

Дополнительная заработная плата — 20%.

Основная заработная плата каждого исполнителя определяется по формуле (5.3):

$$ЗП_{\text{осн}} = Т_{\text{бтс}} \cdot \frac{Т_{\text{к}}}{22} \cdot \Phi_{\text{рв}} \cdot К_{\text{пр}}, \quad (5.3)$$

где $T_{\text{ст}1\text{р}}$ – месячная тарифная ставка 1 разряда;

$T_{\text{к}}$ – тарифный коэффициент согласно разряду исполнителя;

22 – среднее количество рабочих дней;

$\Phi_{\text{рв}}$ – фонд рабочего времени исполнителя;

$K_{\text{пр}}$ – коэффициент премии.

Дополнительная заработная плата рассчитывается по формуле (5.4):

$$ЗП_{\text{доп}} = ЗП_{\text{осн}} \cdot \frac{Н_{\text{доп. зп}}}{100\%}, \quad (5.4)$$

Результаты вычислений внесём в таблицу 5.2.

Таблица 5.2 – Расчет заработной платы

Категория работников	Разряд	Тарифны коэффициент ($T_{\text{к}}$)	$\Phi_{\text{эф.р.в.}}$ (дн.)	Коэффициент премии ($K_{\text{пр}}$)	Заработная плата, бел.руб.		
					Основная	Дополнительна	Всего
Инженер-программист	9	1,68	90	1,5	3061,80	612,36	3674,16
Инженер-программист	9	1,68	90	1,5	3061,80	612,36	3674,16
Научный руководитель	14	2,31	90	1,5	4209,98	842,00	5051,98
Всего	—	—	—	—	10333,58	2066,72	12400,30

Таким образом, заработная плата инженеров-программистов составляет 3674,16 бел. руб. на каждого, а заработная плата научного руководителя составляет 5051,98 бел. руб.

Отчисления на социальные нужды определяются по формуле (5.5):

$$P_{\text{соц}} = (3П_{\text{осн}} + 3П_{\text{доп}}) \cdot \frac{35\%}{100\%}, \quad (5.5)$$

Отчисления на социальные нужды определяются по формуле (5.5):

$$P_{\text{соц}} = (10\,333,58 + 2\,066,72) \cdot \frac{35\%}{100\%} = 4\,340,10 \text{ (белорусских рубля)}.$$

Расходы по статье «Спецоборудование» не рассчитываются, так как приобретение специального оборудования для разработки программного продукта не предусматривалось.

Расходы по статье «Материалы и комплектующие изделия» не рассчитываются, так как материалы в рамках проекта не приобретались.

Расходы по статье «Машинное время» определяются по формуле (5.6):

$$P_{\text{мв}} = Ц_{\text{мв}} \cdot \frac{V_0}{100} \cdot H_{\text{мв}}, \quad (5.6)$$

где $Ц_{\text{мв}}$ – цена одного машино-часа, 7 бел. руб.;

V_0 – уточнённый общий объём программного продукта;

$H_{\text{мв}}$ – норматив расхода машинного времени на отладку 100 строк кода, 0,7 машино-часа.

Рассчитаем расходы на машинное время

$$P_{\text{мв}} = 7 \cdot \frac{10780}{100} \cdot 0,7 = 528,22 \text{ (белорусских рубля)}.$$

V_0 – уточнённый общий объём программного продукта;

Расходы по статье «Научные командировки» не рассчитываются, так как командировки в рамках проекта не предусматривались.

Расходы по статье «Прочие затраты» не рассчитываются, так как специальная научно-техническая информация и специальная литература не приобретались.

Накладные расходы рассчитываются по формуле (5.7):

$$P_{\text{нр}} = \frac{H_{\text{нр}}}{100\%} \cdot 3П_{\text{осн}}, \quad (5.7)$$

где $H_{\text{нр}}$ – норматив накладных расходов.

В данном дипломном проекте норматив накладных расходов равен 30%, поэтому затраты на накладные расходы равны:

$$P_{\text{нр}} = \frac{30\%}{100\%} \cdot 10\,333,58 = 3\,100,07 \text{ (белорусских рублей)}.$$

Сумма вышеперечисленных расходов по статьям СЗ на программный продукт служит исходной базой для расчёта затрат на освоение и сопровождение

программного продукта. Они рассчитываются по формуле (5.8).

$$C3 = 3П_{\text{осн}} + 3П_{\text{доп}} + P_{\text{соц}} + P_{\text{м}} + P_{\text{с}} + P_{\text{мв}} + P_{\text{нк}} + P_{\text{пр}} + P_{\text{нр}}, \quad (5.8)$$

Тогда, $C3 = 3П_{\text{осн}} + 3П_{\text{доп}} + P_{\text{соц}} + P_{\text{м}} + P_{\text{с}} + P_{\text{мв}} + P_{\text{нк}} + P_{\text{пр}} + P_{\text{нр}} = 20368,70$ (белорусских рубля).

Организация–разработчик участвует в освоении программного продукта и несёт соответствующие затраты, на которые составляется смета, оплачиваемая заказчиком по договору. Затраты на освоение P_o определяются по установленному нормативу от суммы затрат по формуле (5.9).

$$P_o = C3 \cdot \frac{H_o}{100\%}, \quad (5.9)$$

где $C3$ – сумма вышеперечисленных расходов по статьям на разработку программного продукта;

H_o – установленный норматив затрат на освоение. Для данного дипломного проекта принимается равной 5%.

Рассчитаем затраты на освоение продукта:

$$P_o = 20\,368,70 \cdot \frac{5\%}{100\%} = 1\,018,44 \text{ (белорусских рубля)}$$

Организация–разработчик осуществляет сопровождение программного продукта и несёт расходы, которые оплачиваются заказчиком в соответствии с договором и сметой на сопровождение. Расходы на сопровождение P_{co} рассчитываются по формуле (5.10).

$$P_{co} = C3 \cdot \frac{H_{co}}{100\%}, \quad (5.10)$$

где H_{co} – установленный норматив затрат на сопровождение программного продукта. Для данного дипломного проекта принимается равным 5%.

Рассчитаем расходы на сопровождение:

$$P_{co} = 20\,368,70 \cdot \frac{5\%}{100\%} = 1\,018,44 \text{ (белорусских рубля)}.$$

Полная себестоимость (СП) разработки программного продукта рассчитывается как сумма расходов по всем статьям. Она определяется по формуле (5.12).

$$СП = C3 + P_o + P_{co}, \quad (5.11)$$

Рассчитаем полную себестоимость продукта:

$$СП = 20\,368,70 + 1\,018,44 + 1\,018,44 = 22\,405,58 \text{ (белорусских рубля)}$$

Результаты вычислений занесём в таблицу 5.3.

Таблица 5.3 – Себестоимость программного продукта

Наименование статей затрат	Норматив, %	Сумма затрат, бел.руб.
Заработная плата, всего	—	12400,30
Основная заработная плата	—	10333,58
Дополнительная заработная плата	20	2066,72
Отчисления на социальные нужды	35	4340,11
Спецоборудование	Не применялось	—
Материалы	Не применялись	—
Машинное время	—	528,22
Научные командировки	Не планировались	—
Прочие затраты	Не применялись	—
Накладные расходы	30	3100,07
Сумма затрат	—	20368,70
Затраты на освоение	5	1018,44
Затраты на сопровождение	5	1018,44
Полная себестоимость	—	22405,58

В результате всех расчётов полная себестоимость программного продукта составила 22 405,58 бел. руб.

5.4 Расчет отпускной цены и чистой прибыли

Для определения цены программного продукта необходимо рассчитать плановую прибыль П, которая рассчитывается по формуле (5.12).

$$\Pi = \text{СП} \cdot \frac{R}{100\%}, \quad (5.12)$$

где СП – полная себестоимость программного модуля, бел. руб; R – уровень рентабельности программного модуля.

В данном дипломном проекте уровень рентабельности равен 30%. Рассчитаем прибыль от реализации:

$$\Pi = 22\,405,58 \cdot \frac{30\%}{100\%} = 6\,721,67 \text{ (белорусских рубля)}$$

После расчета прибыли от реализации по формуле (5.13) определяется прогнозируемая цена программного продукта без налогов Ц_п.

$$\text{Ц}_\text{п} = \text{СП} + \Pi, \quad (5.13)$$

Рассчитаем цену программного продукта без налогов:

$$\text{Ц}_\text{п} = 22\,405,58 + 6\,721,67 = 29\,127,25 \text{ (белорусских рубля)}.$$

Отпускная цена Ц_о (цена реализации) программного продукта включает налог на добавленную стоимость и рассчитывается по формуле (5.14).

$$\text{Ц}_\text{о} = \text{СП} + \Pi + \text{НДС}_\text{пп}, \quad (5.14)$$

где НДС_{пп} – налог на добавленную стоимость для программного продукта. Для данного программного продукта НДС_{пп} рассчитывается по формуле (5.15).

$$\text{НДС}_\text{пп} = \text{Ц}_\text{п} \cdot \frac{\text{НДС}}{100\%}, \quad (5.15)$$

где НДС – налог на добавленную стоимость. В настоящее время он составляет 20%.

Рассчитаем НДС_{пп} и отпускную цену:

$$\text{НДС}_\text{пп} = 29\,127,25 \cdot \frac{20\%}{100\%} = 5\,825,45 \text{ (белорусских рубля)}.$$

$$\text{Ц}_\text{о} = 22\,405,58 + 6\,721,67 + 5\,825,45 = 34\,952,70 \text{ (белорусских рубля)}.$$

Прибыль от реализации программного продукта за вычетом налога на прибыль является чистой прибылью ПЧ. Чистая прибыль остаётся организации–разработчику и представляет собой экономический эффект от создания нового программного продукта. Она рассчитывается по формуле (5.16).

$$\text{ПЧ} = \Pi \cdot \frac{\text{Н}_\text{п}}{100\%}, \quad (5.16)$$

где Н_п – ставка налога на прибыль. В настоящее время он равен 20%. Рассчитаем чистую прибыль:

$$ПЧ = 6\,721,67 \cdot \left(1 - \frac{20\%}{100\%}\right) = 5\,377,34 \text{ (белорусских рубля)}.$$

Результаты расчётов цены и прибыли по программному продукту сведены в таблицу 5.4.

Таблица 5.4. – Расчёт отпускной цены и чистой прибыли программного модуля

Наименование статей затрат	Норматив, %	Сумма затрат, бел. руб.
Полная себестоимость	—	22405,58
Прибыль	30	6721,67
Цена без НДС	—	29127,25
НДС	20	5825,45
Отпускная цена	—	34952,70
Налог на прибыль	20	1344,33
Чистая прибыль	—	5377,34

В ходе произведенных расчетов определены основные экономические показатели: полная себестоимость — 22405,58 бел. руб., прогнозируемая цена — 29127,25 бел. руб., чистая прибыль — 5377,34 бел. руб.

Разработанный программный продукт представляет собой комплексную e-commerce систему, включающую мобильное приложение, серверную часть, административную панель, модуль оплаты и интеллектуальный подбор товаров. Таким образом, рассчитанная отпускная цена программного продукта, разрабатываемого в рамках данного дипломного проекта, составляет 34952,70 белорусских рубля. При расчете цены учтены отчисления в фонд социальной защиты, машинное время, накладные расходы, затраты на освоение и сопровождение, а также налоги, необходимые к уплате.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы была спроектирована и разработана система электронной коммерции с акцентом на мобильное приложение для покупателей. Основное внимание было уделено созданию удобного инструмента для просмотра каталога товаров, поиска подходящей продукции, добавления товаров в корзину, оформления заказа и взаимодействия пользователя с онлайн-магазином.

В процессе работы был проведён анализ предметной области электронной коммерции, рассмотрены особенности современных мобильных торговых приложений и решений, использующих интеллектуальный подбор товаров. На основе проведённого анализа была сформулирована задача разработки системы, включающей мобильный клиент, серверную часть и административную панель.

В рамках проектирования была определена общая архитектура системы, разработана структура базы данных, описана логика взаимодействия между основными компонентами, а также обоснован выбор используемых технологий. Серверная часть обеспечивает обработку пользовательских запросов, хранение данных, работу с заказами, авторизацией и внешними сервисами. Административная панель предоставляет возможность управления товарами, клиентами и заказами.

Особое внимание в работе было уделено модулю интеллектуального подбора товаров. Данный модуль позволяет пользователю формулировать запросы в свободной форме и получать более релевантные рекомендации по сравнению с обычным поиском и фильтрацией. Это повышает удобство использования приложения и делает процесс выбора товаров более персонализированным.

Также было проведено тестирование основных компонентов системы: мобильного приложения, серверной части, административной панели и модуля интеллектуального подбора товаров. По результатам тестирования было установлено, что разработанная система выполняет заявленные функции и может использоваться как основа для дальнейшего развития.

Таким образом, цель дипломной работы была достигнута. Разработанная система демонстрирует практическую возможность создания мобильного приложения электронной коммерции с интеллектуальным подбором товаров, а также обладает потенциалом для дальнейшего расширения функциональности, масштабирования каталога и улучшения пользовательского опыта.

СПИСОК СОКРАЩЕНИЙ

AI – Artificial Intelligence (искусственный интеллект).

API – Application Programming Interface (интерфейс программирования приложений).

REST – Representational State Transfer (архитектурный стиль взаимодействия клиентской и серверной частей).

HTTP – Hypertext Transfer Protocol (протокол передачи гипертекста).

HTTPS – Hypertext Transfer Protocol Secure (защищённый протокол передачи гипертекста).

JSON – JavaScript Object Notation (текстовый формат обмена данными).

JWT – JSON Web Token (токен для передачи данных авторизации).

URL – Uniform Resource Locator (унифицированный указатель ресурса).

UI – User Interface (пользовательский интерфейс).

UX – User Experience (пользовательский опыт).

MVP – Minimum Viable Product (минимально жизнеспособный продукт).

LLM – Large Language Model (большая языковая модель).

SSO – Single Sign-On (единый вход в систему).

SDK – Software Development Kit (набор средств разработки программного обеспечения).

CRUD – Create, Read, Update, Delete (создание, чтение, обновление и удаление данных).

БД – база данных.

СУБД – система управления базами данных.

ODM – Object Document Mapper (средство объектно-документного отображения данных).

NoSQL – Not Only SQL (подход к организации нереляционных баз данных).

JS – JavaScript (язык программирования JavaScript).

TS – TypeScript (язык программирования TypeScript).

HTML – HyperText Markup Language (язык гипертекстовой разметки).

CSS – Cascading Style Sheets (каскадные таблицы стилей).

NPM – Node Package Manager (менеджер пакетов для Node.js).

CLI – Command Line Interface (интерфейс командной строки).

ENV – Environment Variable (переменная окружения).

QA – Quality Assurance (обеспечение качества программного обеспечения).

IDE – Integrated Development Environment (интегрированная среда разработки).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Amazon's next-gen AI assistant for shopping is now even smarter, more capable, and more helpful [Электронный ресурс]. – Режим доступа: <https://www.aboutamazon.com/news/retail/amazon-rufus-ai-assistant-personalized-shopping-features>. – Дата доступа: 07.04.2026.
2. Shopping on Google: AI Mode and virtual try-on updates from I/O 2025 [Электронный ресурс]. – Режим доступа: <https://blog.google/products-and-platforms/products/shopping/google-shopping-ai-mode-virtual-try-on-update/>. – Дата доступа: 07.04.2026.
3. eBay Uses Agentic AI to Supercharge Personalized Ecommerce [Электронный ресурс]. – Режим доступа: <https://innovation.ebayinc.com/stories/ebay-uses-agentic-ai-to-supercharge-personalized-ecommerce/>. – Дата доступа: 08.04.2026.
4. React Native Documentation [Электронный ресурс]. – Режим доступа: <https://reactnative.dev/docs/getting-started>. – Дата доступа: 08.04.2026.
5. Express Documentation [Электронный ресурс]. – Режим доступа: <https://expressjs.com/>. – Дата доступа: 09.04.2026.
6. MongoDB Documentation [Электронный ресурс]. – Режим доступа: <https://www.mongodb.com/docs/>. – Дата доступа: 09.04.2026.
7. Gemini API Documentation [Электронный ресурс]. – Режим доступа: <https://ai.google.dev/gemini-api/docs>. – Дата доступа: 10.04.2026.
8. Expo Documentation [Электронный ресурс]. – Режим доступа: <https://docs.expo.dev/>. – Дата доступа: 10.04.2026.
9. Expo Router Documentation [Электронный ресурс]. – Режим доступа: <https://docs.expo.dev/router/introduction/>. – Дата доступа: 11.04.2026.
10. TanStack Query Documentation [Электронный ресурс]. – Режим доступа: <https://tanstack.com/query/v5/docs/framework/react/react-native>. – Дата доступа: 11.04.2026.
11. Clerk Expo SDK Documentation [Электронный ресурс]. – Режим доступа: <https://clerk.com/docs/reference/expo/overview>. – Дата доступа: 12.04.2026.
12. Stripe React Native SDK Documentation [Электронный ресурс]. – Режим доступа: <https://docs.stripe.com/sdks/react-native>. – Дата доступа: 12.04.2026.
13. NativeWind Documentation [Электронный ресурс]. – Режим доступа: <https://www.nativewind.dev/>. – Дата доступа: 13.04.2026.
14. Sentry React Native Documentation [Электронный ресурс]. – Режим доступа: <https://docs.sentry.io/platforms/react-native/>. – Дата доступа: 13.04.2026.
15. Vercel Documentation [Электронный ресурс]. – Режим доступа: <https://vercel.com/docs/frameworks/backend/express>. – Дата доступа: 14.04.2026.